零代码实战全栈多端开发

0 代码全栈开发 SpringBoot+uniapp 商城 APP

Figma+Springboot+Vue3+Uniapp 商城 APP AI 编程 0 代码实战 全栈开发 保姆级教程

全栈多端开发：

原型开发（Figma） + 数据库（Mysql） + 后端（Springboot） + PC 前端（vue3） + Uniapp 移动端（小程序前端+app 前端）+ 前后联调

课程特色:

不写一行代码，从 0 到 1 完整开发一个 java 商城小程序/APP。技术全面而系统，包含完整 APP 开发流程，涉及多个核心技术，多开发平台的联合使用与调试，学完后具有开发后台+APP+小程序多端全栈的基本能力。

Uniapp 是一个使用 Vue.js 开发所有前端应用的框架。开发者编写一套代码，可发布到 iOS、Android、鸿蒙 Next、Web（响应式）、以及各种小程序（微信/支付宝/百度/抖音/飞书/QQ/快手/钉钉/淘宝/京东/小红书）、快应用、鸿蒙元服务等多个平台。终极跨平台解决方案。

技术栈：

原型设计：Figma

数据库：Mysql

后端：Java+Springboot+Mybatis-plus+Maven+jwt

PC 前端：Vue3 + Vue Router + Pinia + Element Plus + Axios + Vite

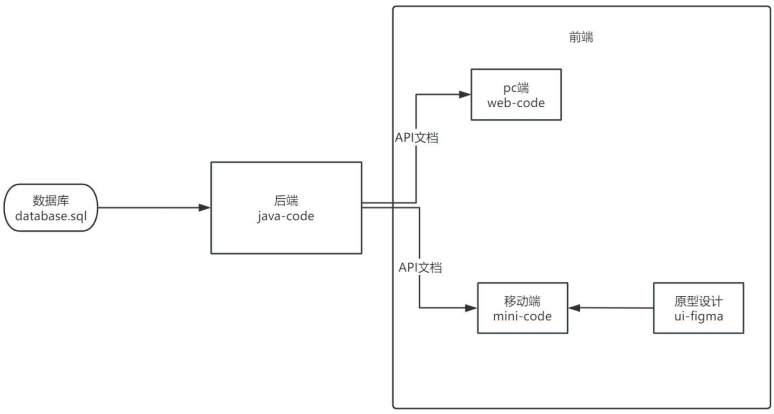
移动端：Uniapp

开发工具：IDEA + Navicat + Hbuilder + 微信开发者工具 + AI 编程工具（Cursor、Codebuddy、Trae、Kiro 等不做任何限制）

课程内容

效果演示

一、创建工作区目录



二、移动端前端原型设计

（1）Figma 提示词

1. 第一步先自己总结功能模块

移动端电商商城 APP 原型图，现在需要输出高保真的原型图，底部 tabbar 有：首页、分类、购物车、我的。一共四个 tabbar 。

主要页面：

首页 - 顶部搜索框、轮播图展示促销活动、热门商品列表（网格布局）
分类页 - 左侧分类导航、右侧对应商品列表，点击切换不同分类
购物车页 - 展示已添加商品、可调整数量、删除商品、单选和全选商品、点击结算进入订单页
我的页面 - 个人信息、我的订单（待发货，待收货，已完成，退款/售后）、收货地址、退出登录、注册登录等功能入口。
商品详情页 - 展示商品详情信息、底部加入购物车或立即购买按钮、选择规格弹窗、购物车弹窗、商品评价列表
订单确认页 - 显示收货地址、商品清单、价格明细、支付方式选择
我的订单页 - 显示待发货，待收货，已完成，退款/售后不同状态订单

响应式布局仅支持移动端

参考补充：
☑ 首页 - 搜索栏、轮播图、热门商品列表
☑ 分类 - 左侧分类导航、右侧商品列表、规格选择弹窗
☑ 购物车 - 商品列表、数量控制、全选/单选、结算功能
☑ 个人中心 - 用户信息、订单统计、功能菜单、退出登录
☑ 商品详情 - 商品信息、图文详情、评价展示、规格选择弹窗
☑ 订单确认 - 收货地址、商品清单、价格明细、支付方式
☑ 我的订单 - 订单列表、状态筛选、评价弹窗
☑ 地址管理 - 地址列表、新增/编辑/删除地址
☑ 登录 - 账号密码登录
☑ 注册 - 账号密码注册
☑ 个人信息 - 头像、昵称编辑

2.第二步给到 deepseek 进行润色

【第一步内容】
我需要将上面提示词给到 figma 进行原型设计，帮我润色下提示词

润色后结果：

项目需求：移动端电商商城 APP 高保真原型图 一、设计范围 响应式移动端布局（基于 375×667 或类似常见移动端分辨率） 二、底部 Tab Bar（4 项）

首页 | 分类 | 购物车 | 我的

三、主要页面与功能说明

首页

顶部：搜索框

中部：轮播图（促销活动展示）

下部：热门商品列表（网格布局，每行 2-3 个商品）

分类页

左侧：分类导航（垂直列表，点击切换）

右侧：对应分类的商品列表（网格或列表布局）

购物车页

商品列表：展示已添加商品（含图片、名称、价格、数量）

操作功能：调整数量、删除、单选/全选

底部栏：显示总价与结算按钮（点击进入订单确认页）

我的页面

顶部：个人信息区域

中部功能入口：我的订单（待发货、待收货、已完成、退款/售后）、收货地址管理、设置等

底部：退出登录/注册登录入口

商品详情页

商品图片与基本信息

商品详情展示（图文描述）

底部操作栏：加入购物车、立即购买（点击后弹出规格选择窗口或购物车弹窗）

订单确认页

收货地址选择/编辑

商品清单与价格明细

支付方式选择（如支付宝、微信支付等）

提交订单按钮

我的订单页

订单状态分类标签：待发货、待收货、已完成、退款/售后

订单列表（可点击进入订单详情）

四、交互与状态说明

点击 Tab 可切换页面

分类页左侧导航与右侧内容联动

购物车页支持选中状态与价格实时更新

商品详情页底部操作需触发弹窗（规格选择、购物车浮层）

订单状态页支持横向滑动或标签切换

五、设计交付要求

高保真原型，接近最终 UI 效果

可交互原型（可通过 Figma Prototype 设置基本跳转）

注明主要交互状态（如按钮点击、切换选中）

不断检查和修复页面就可以

（2）AI 编程工具提示词

在 ui-code 文件夹里面， 根据数据库 database.sql 生成高保真的移动端商城 APP 的原型图，每个界面作为独立的 html 文件存放，点击对应功能可以实现页面的跳转。确保这些原型界面可以直接用于开发。

三、创建数据库

（1）提示词

@database.sql　帮我创建 mysql 数据库的管理员表、商品表、用户表、订单表、分类表、轮播图表、评价表

商品表有以下字段：ID、商品名称、封面图、详情图（jsonArray 格式）、价格、原价、库存、状态、是否推荐、销量、浏览量、商品描述、分类、规格（json 格式）

用户表有以下字段：ID、头像、昵称、账号、密码、微信 openid

管理员表有以下字段：ID、账号、密码、昵称、角色、头像

分类表有以下字段：ID、分类名称、分类图片、排序、是否首页展示

订单表有以下字段：ID、用户 ID、订单编号、订单金额、订单信息(jsonArray 格式)、订单备注、订单状态（ 0 待支付, 1 待发货, 2 待收货, 3 已完成, 4 退款/售后）、订单金额、优惠金额、支付金额、联系人、联系电话、收货地址

轮播图表有以下字段：ID、标题、封面图、关联商品 ID

评价表有以下字段：ID、用户 ID、订单 ID、商品 ID、评分、评价内容、评价图片（jsonArray 格式）

每张表都包含：

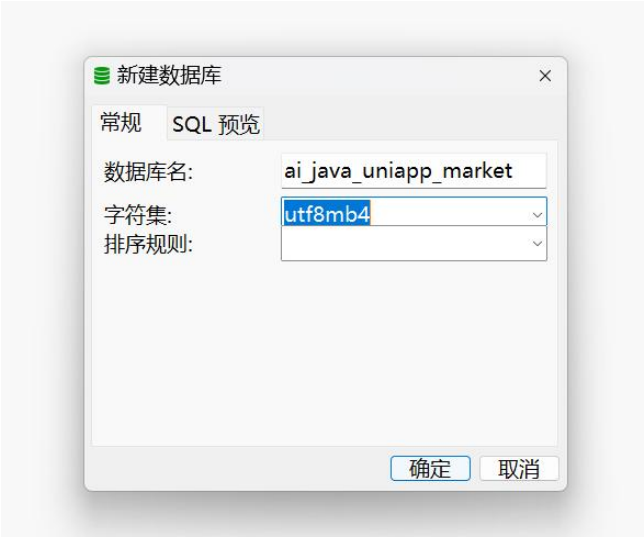
自增主键 ID
create_time 创建时间
update_time 更新时间（自动更新）
is_deleted 删除标记（逻辑删除）

检查：检查所有 jsonArray 字段都为 text 类型。

（2）导入 mysql

下载 mysql、navicat

创建数据库 名称 ai_java_uniapp_market



四、创建 pc 前端工程（后台管理）和原型开发

（1）提示词

在当【@web-code】下面，请根据数据库文件【@xxx.sql】内容帮我创建 xxx 后台管理的前端工程，然后设计后台管理系统的前端原型，要求如下：

1.前端框架采用 vue3,路由管理采用 Vue Router,状态管理采用 Pinia ，UI 框架采用 Element Plus,与后端交互采用 axios,项目构建工具使用 vite。
2.实现页面包括管理员账号密码登录页面（为登录页面添加了临时测试模式,在没有后端 API 的情况下登录测试前端界面）、登录后进入系统，系统左侧是菜单栏，菜单栏显示菜单：【首页、用户管理、xx 数据管理】。
4.api 接口层采用单独目录，不要放入 utils 目录中
5.实现功能时采用分层设计的实现，注意前后连贯一致。
6.只实现页面,具体前端功能暂时不做。实现时页面字段和数据注意与数据库【@数据库文件】保持一致，采用驼峰命名。界面设计时采用扁平化设计,界面简洁，美观，精致，能直接用于后期开发。

（2）运行调试

进入 web-code 终端
npm install
npm run dev

（3）修正页面

布局和样式和功能修正，原型生成后，不断调试和优化页面功能：

为所有页面的新增和编辑按钮添加对话框显示功能

统计图功能：

Home.vue 最近 7 天订单趋势采用 Echarts 折线图来实现，只需要实现页面就可以。删除快速入口功能，快速入口这块显示月交易额统计图，采用 Echarts 柱状图来实现

采用最简单的规格结构，只有规格名称和规格值两个字段，不包含价格和库存信息。所有规格共享商品的基础价格和库存。
规格数据结构：
[
  {
  "name": "颜色",
  "value": "红色"
  },
  {
  "name": "尺寸",
  "value": "L 码"
  }
]

AI 最后做再次检查

请检查下按照【@database.sql】数据库字段模拟数据，字段名称要一致。数据库使用的是下划线命名，前端代码中采用驼峰命名。所有页面的模拟数据现在都严格遵循数据库表结构，字段名称使用驼峰命名。

五、pc 前端（后台管理）接口文档生成

（一）第 1 次提问：生成接口文档

现在请根据原型，生成后端接口文档保存到【API.md】里面。要求如下:

1 分析原型(原型在【/src/view】目录下)中的功能模块，设计出后端接口，并编写后台管理端接口文档，后面会加移动端接口文档内容。

2 接口与原型前端对接，因此前后必须一致。

3 接口风格采用 RESTful 风格，接口必须符合工程规范，专业。

4 每个接口至少包括但不限于接口名称，接口功能，接口访问路径，接口请求方式，接口的请求传递参数，接口返回数据，并要求有使用示范案例。注意一定要专业实用(如果有我没提到但属于专业也加上)。

5.后台管理系统的接口路径统一为 /api/admin/{模块}/{操作}，移动端小程序接口路径格式应为 /api/mini/{模块}/{操作}，这样可以清晰区分 PC 端后台管理和移动端的接口。

（二）第 2 次提问：修正文档

认真检查一下当前的【@接口文档.md】是否完全支持【web-code】原型中的所有功能，因为后端将严格按照接口实现，如果接口不完善，将导致前后无法对接。

注意：删除导出导入接口文档内容,删除对后台 java 的要求文档内容，不需要他在文档里面要求后端怎么实现，只需要专注写明接口文档就可以了。

（三）第 3 次提问：修正文档 驼峰命名

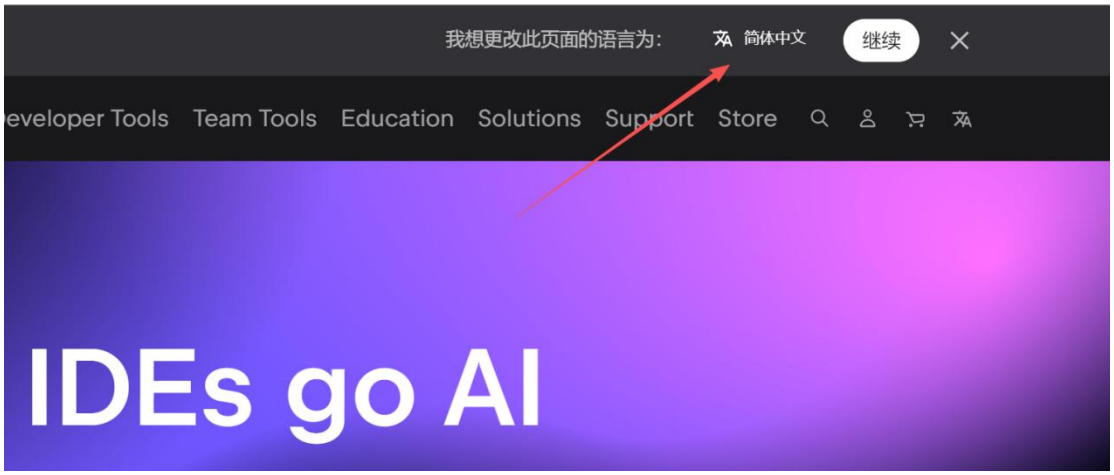
@接口文档.md 我认真检查了一下接口文档，请求参数名字中有很多下划线命名，而后台返回的是驼峰命名，可能导致前后无法对接，请把接口文档中参数(包括请求参数和返回参数)的名字由下划线命名改为驼峰命名。

API.md 后台管理系统的接口路径统一为 /api/admin/{模块}/{操作}，移动端小程序接口路径格式应为 /api/mini/{模块}/{操作}，这样可以清晰区分 PC 端后台管理和移动端的接口。

六、后端工程创建

下载 IDEA

https://lp.jetbrains.com/

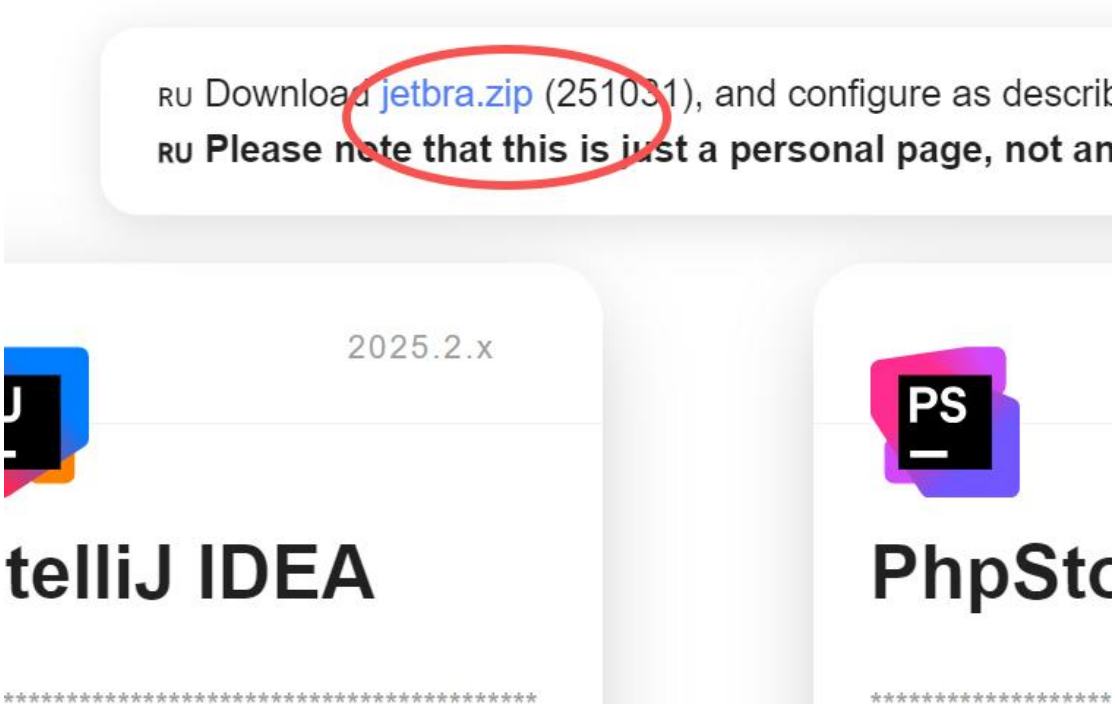


https://www.jetbrains.com.cn/idea/

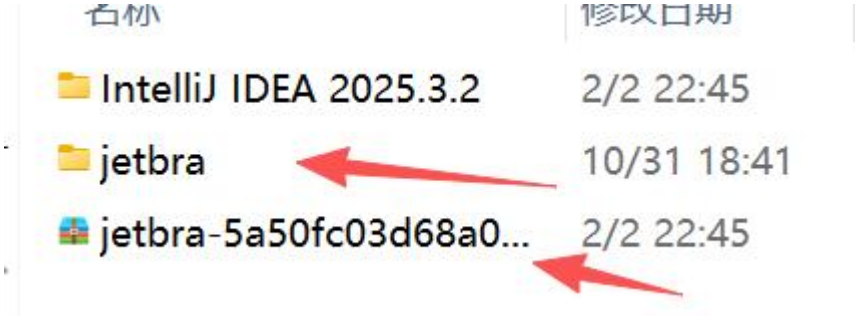
安装

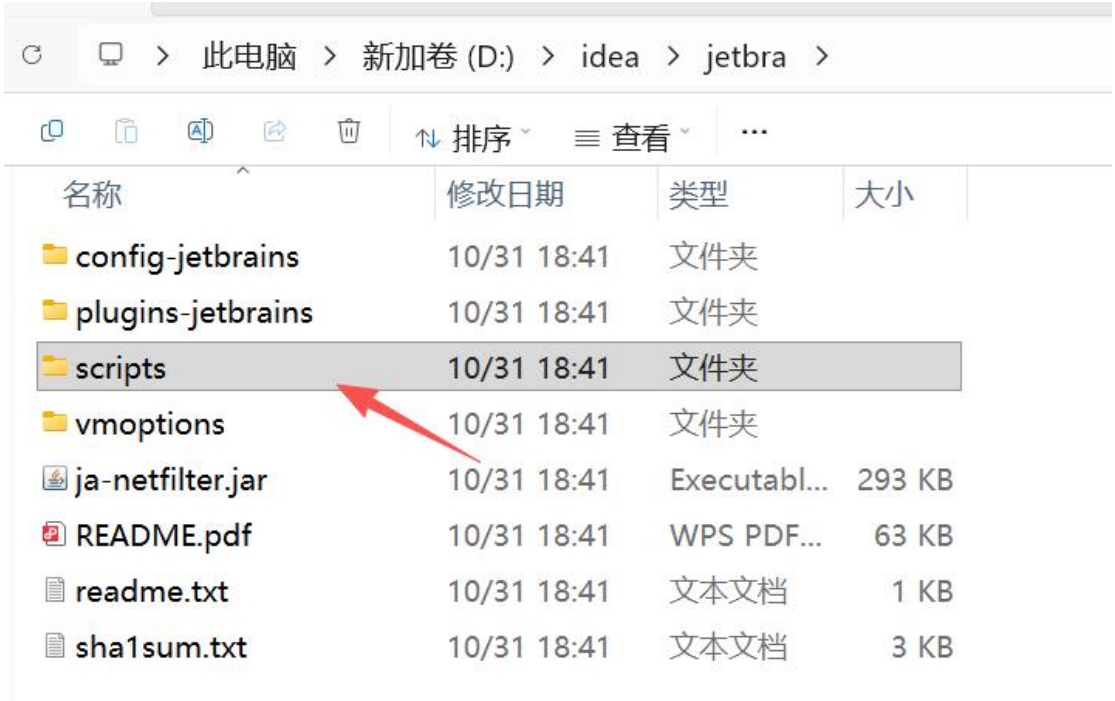
激活码：

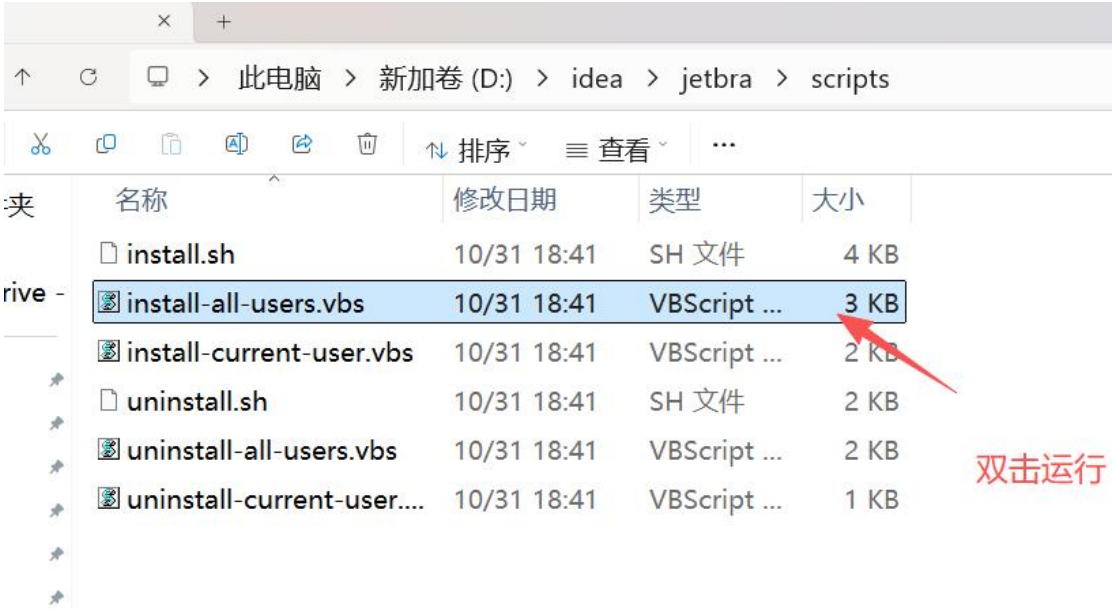
https://3.jetbra.in



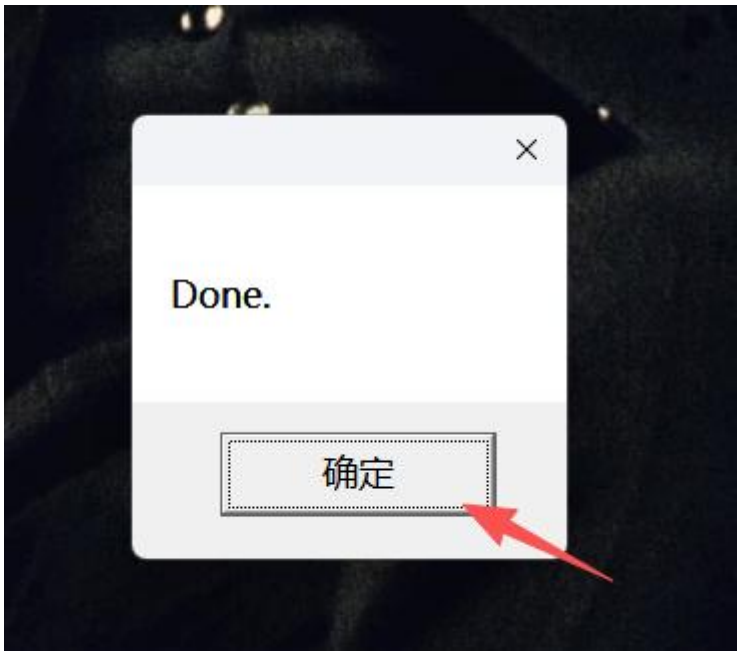
下载下来解压放到 idea 软件相同目录里面





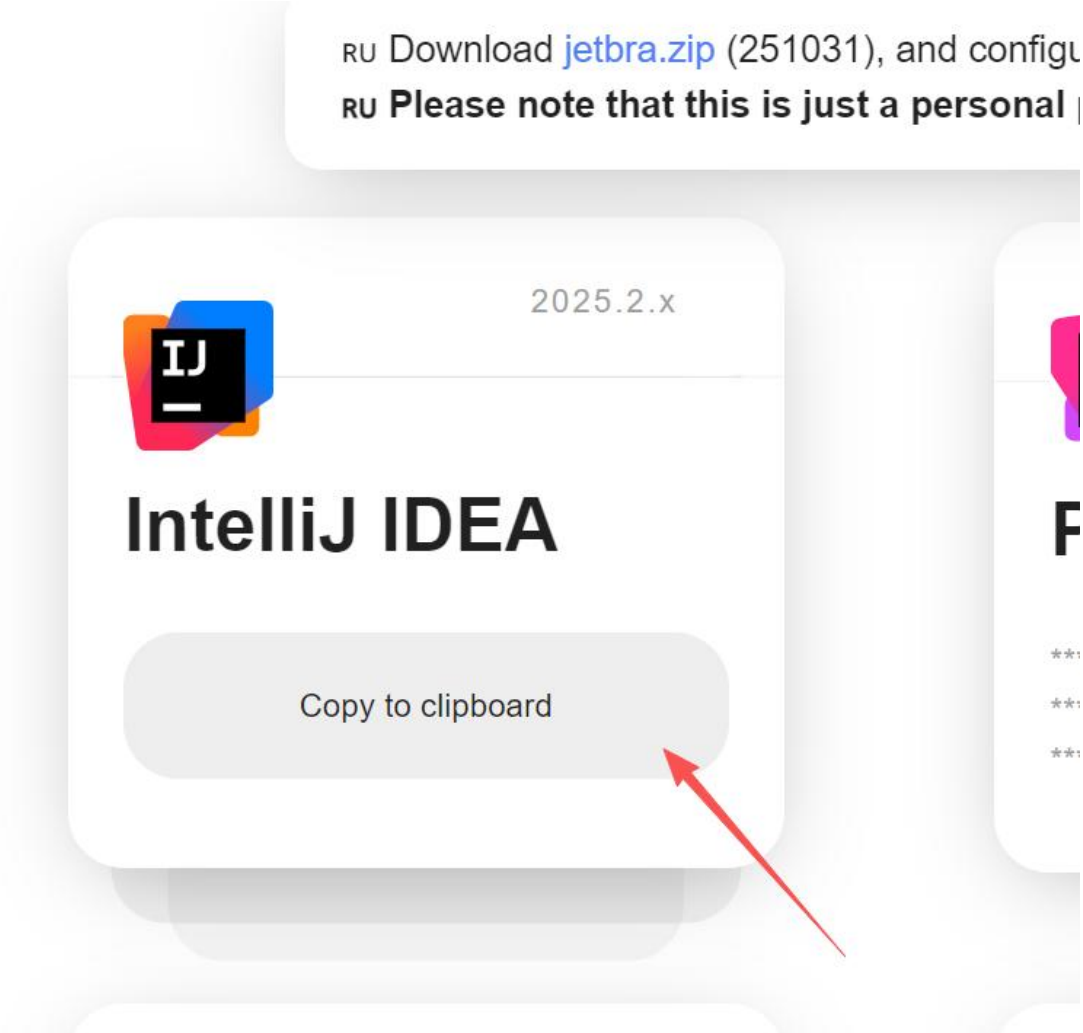


然后点确定，然后稍等几分钟，弹出一个 down 代表完成

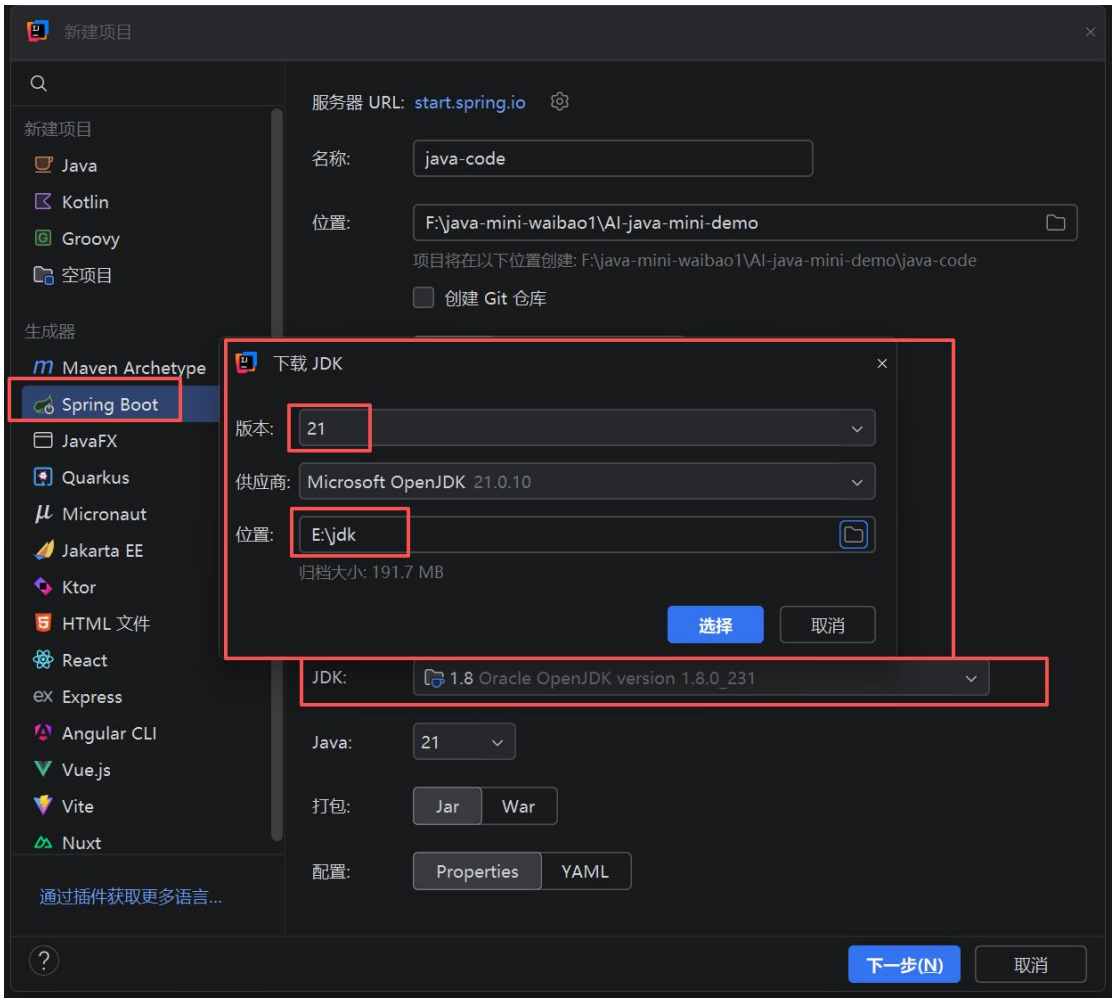

Done.
确定

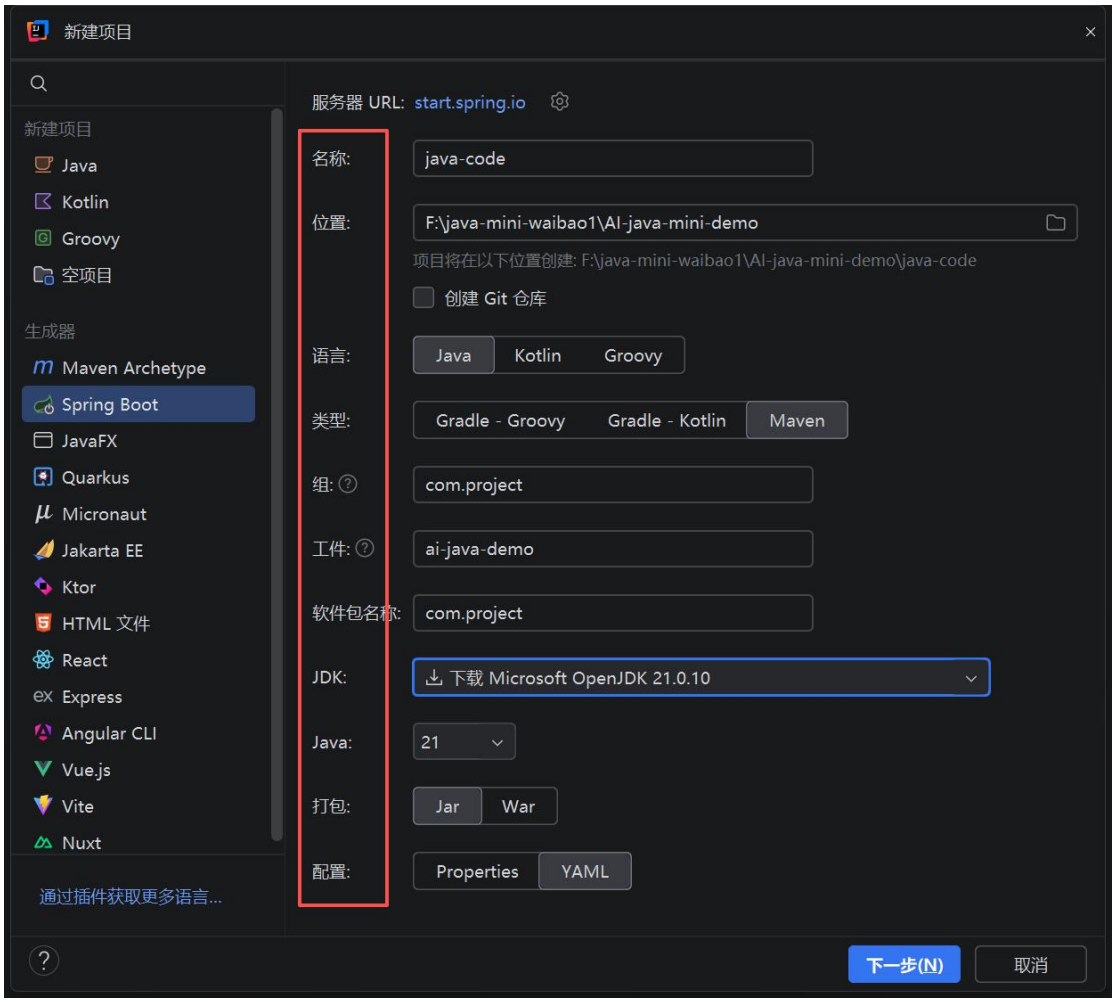
打开 idea，破解

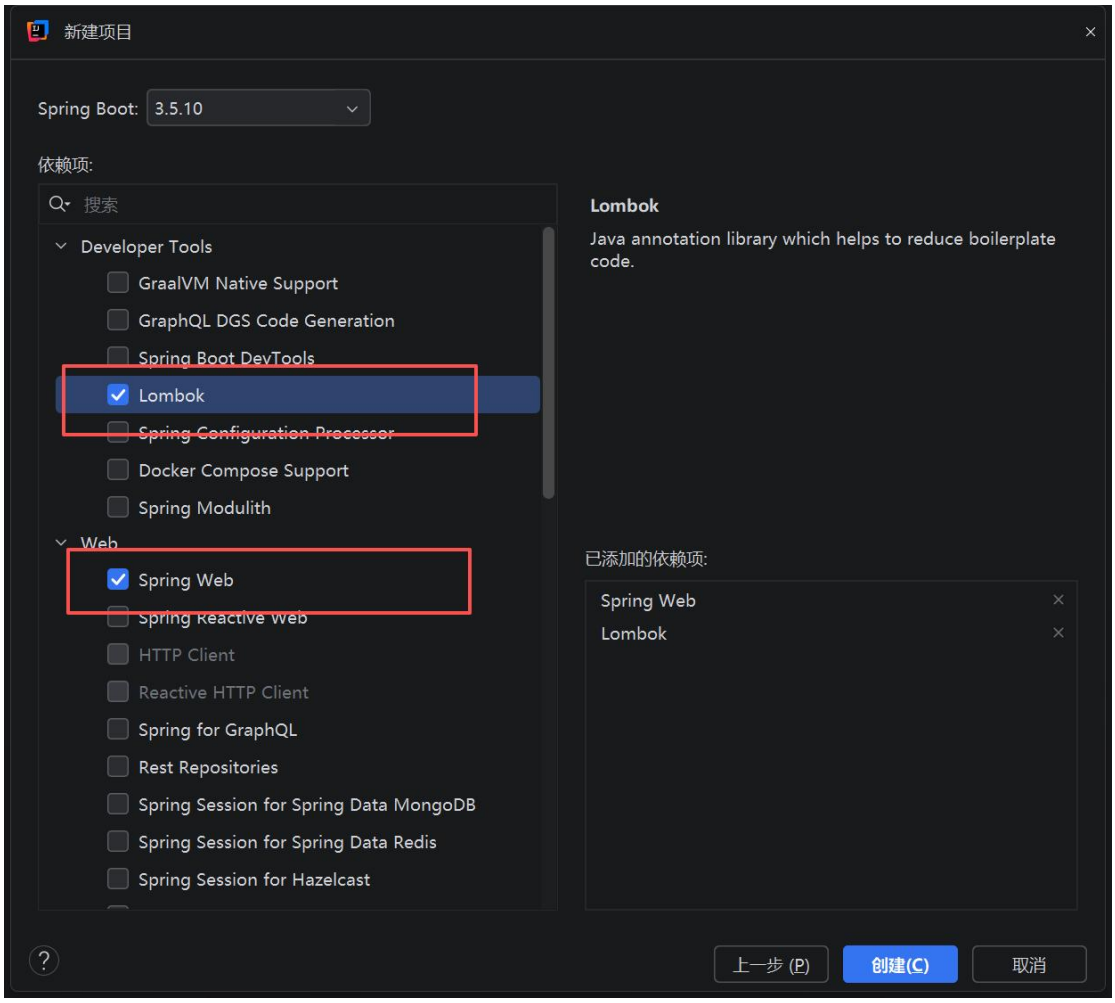
管理订阅
IDE
IntelliJ IDEA
激活
付费插件
Code With Me
激活以启用
IntelliJ IDEA
方案和定价 ↗
无订阅
JetBrains Account
激活码(C)
许可证服务器(S)
在此处粘贴激活码或将其拖放到此处
激活
登录...
关闭


RU Download jetbra.zip (251031), and configu
RU Please note that this is just a personal
2025.2.x
IntelliJ IDEA
Copy to clipboard

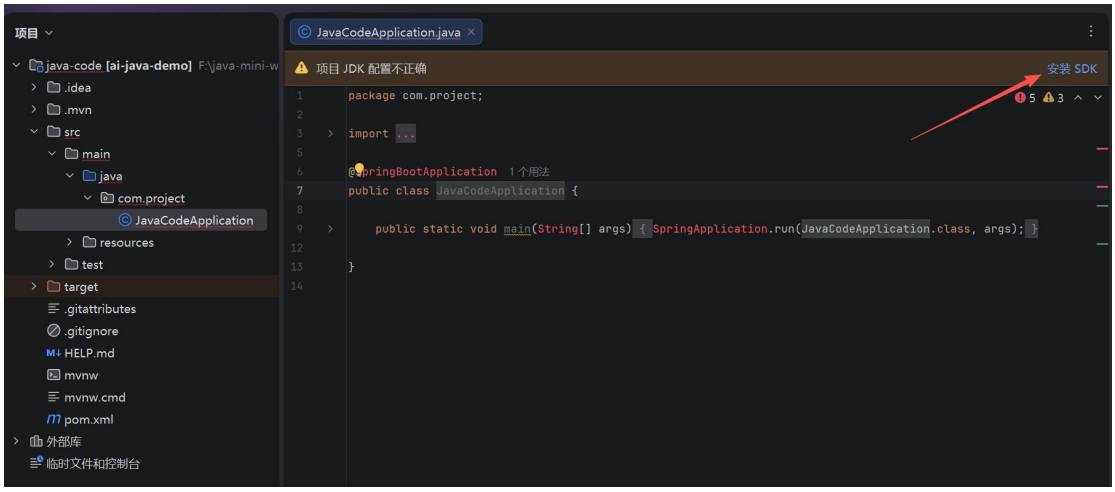
创建项目

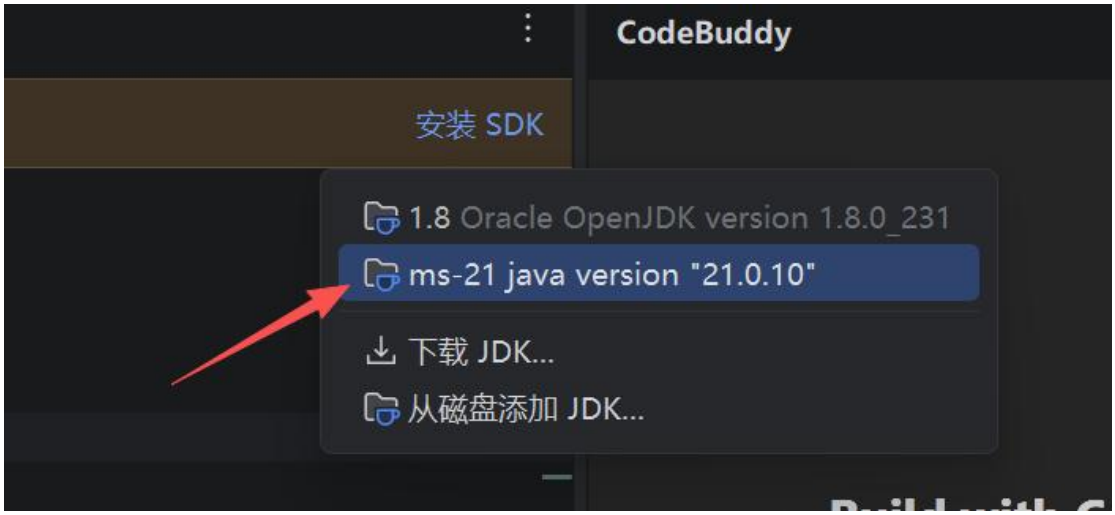

新建项目
新建项目
Java
Kotlin
Groovy
空项目
生成器
Maven Archetype
Spring Boot
JavaFX
Quarkus
Micronaut
Jakarta EE
Ktor
HTML 文件
React
Express
Angular CLI
Vue.js
Vite
Nuxt
通过插件获取更多语言...
服务器 URL: start.spring.io
名称: java-code
位置: F:\java-mini-waibao1\AI-java-mini-demo
项目将在以下位置创建: F:\java-mini-waibao1\AI-java-mini-demo\java-code
创建 Git 仓库
下载 JDK
版本: 21
供应商: Microsoft OpenJDK 21.0.10
位置: E:\jdk
归档大小: 191.7 MB
选择
取消
JDK: 1.8 Oracle OpenJDK version 1.8.0_231
Java: 21
打包: Jar War
配置: Properties YAML
下一步(N)
取消


新建项目
新建项目
Java
Kotlin
Groovy
空项目
生成器
Maven Archetype
Spring Boot
JavaFX
Quarkus
Micronaut
Jakarta EE
Ktor
HTML 文件
React
Express
Angular CLI
Vue.js
Vite
Nuxt
通过插件获取更多语言...
服务器 URL: start.spring.io
名称: java-code
位置: F:\java-mini-waibao1\AI-java-mini-demo
项目将在以下位置创建: F:\java-mini-waibao1\AI-java-mini-demo\java-code
创建 Git 仓库
语言: Java Kotlin Groovy
类型: Gradle - Groovy Gradle - Kotlin Maven
组: com.project
工件: ai-java-demo
软件包名称: com.project
JDK: 下载 Microsoft OpenJDK 21.0.10
Java: 21
打包: Jar War
配置: Properties YAML
下一步(N)
取消

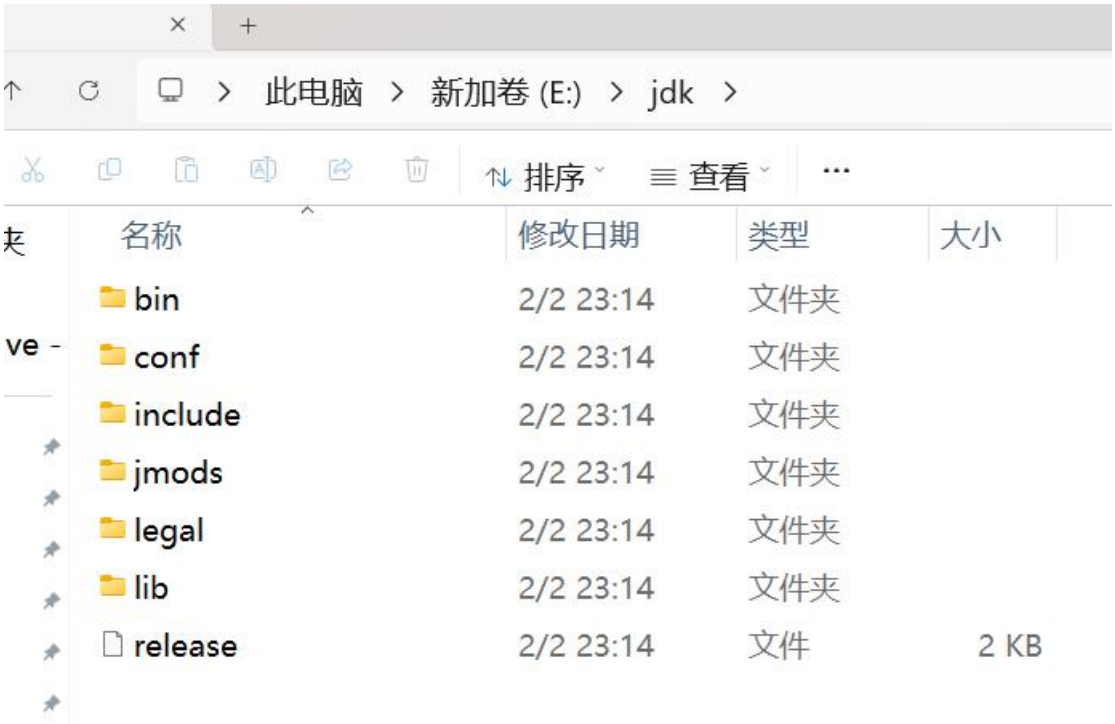

新建项目
Spring Boot: 3.5.10
依赖项:
搜索
Developer Tools
GraalVM Native Support
GraphQL DGS Code Generation
Spring Boot DevTools
Lombok
Spring Configuration Processor
Docker Compose Support
Spring Modulith
Web
Spring Web
Spring Reactive Web
HTTP Client
Reactive HTTP Client
Spring for GraphQL
Rest Repositories
Spring Session for Spring Data MongoDB
Spring Session for Spring Data Redis
Spring Session for Hazelcast
Lombok
Java annotation library which helps to reduce boilerplate code.
已添加的依赖项:
Spring Web
Lombok
上一步 (P)
创建(C)
取消

安装 sdk


项目
java-code [ai-java-demo] F:\java-mini-w
.idea
.mvn
src
main
java
com.project
JavaCodeApplication
resources
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
外部库
临时文件和控制台
JavaCodeApplication.java
项目 JDK 配置不正确
安装 SDK
package com.project;
import ...
@SpringBootApplication 1 个用法
public class JavaCodeApplication {
public static void main(String[] args) { SpringApplication.run(JavaCodeApplication.class, args); }
}

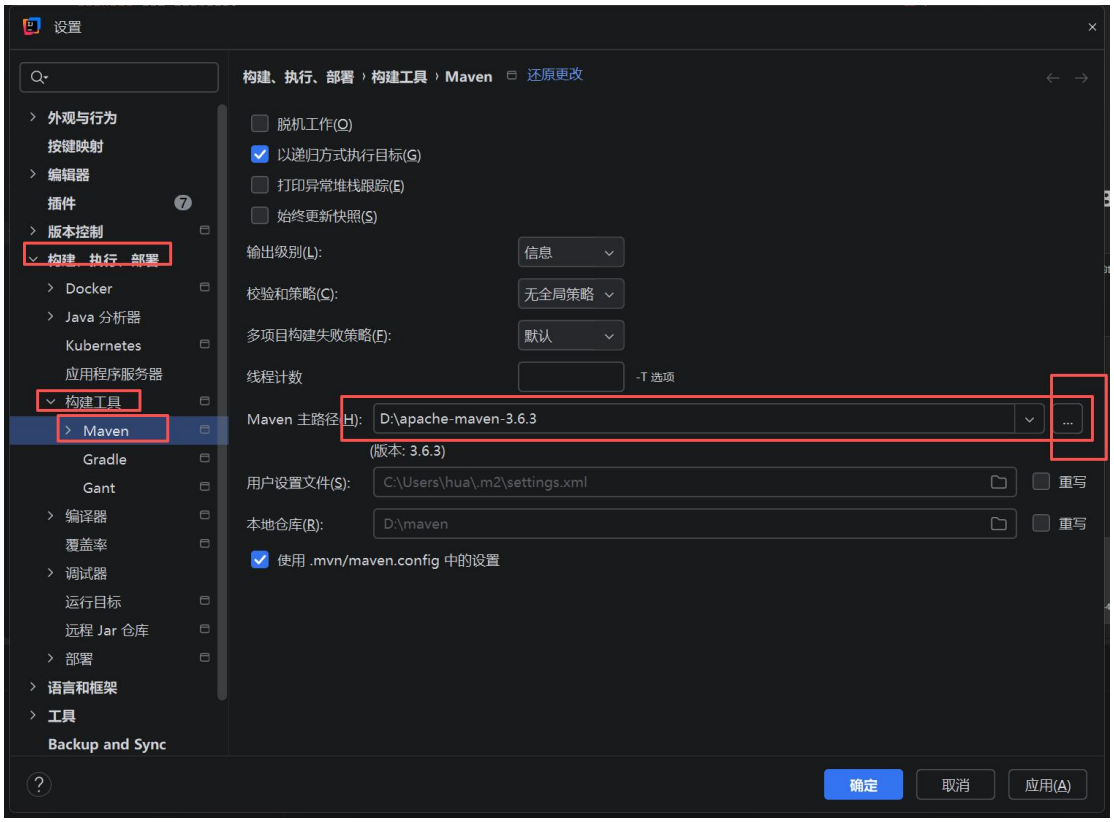


不行的话那就选择你的 jdk 目录就可以



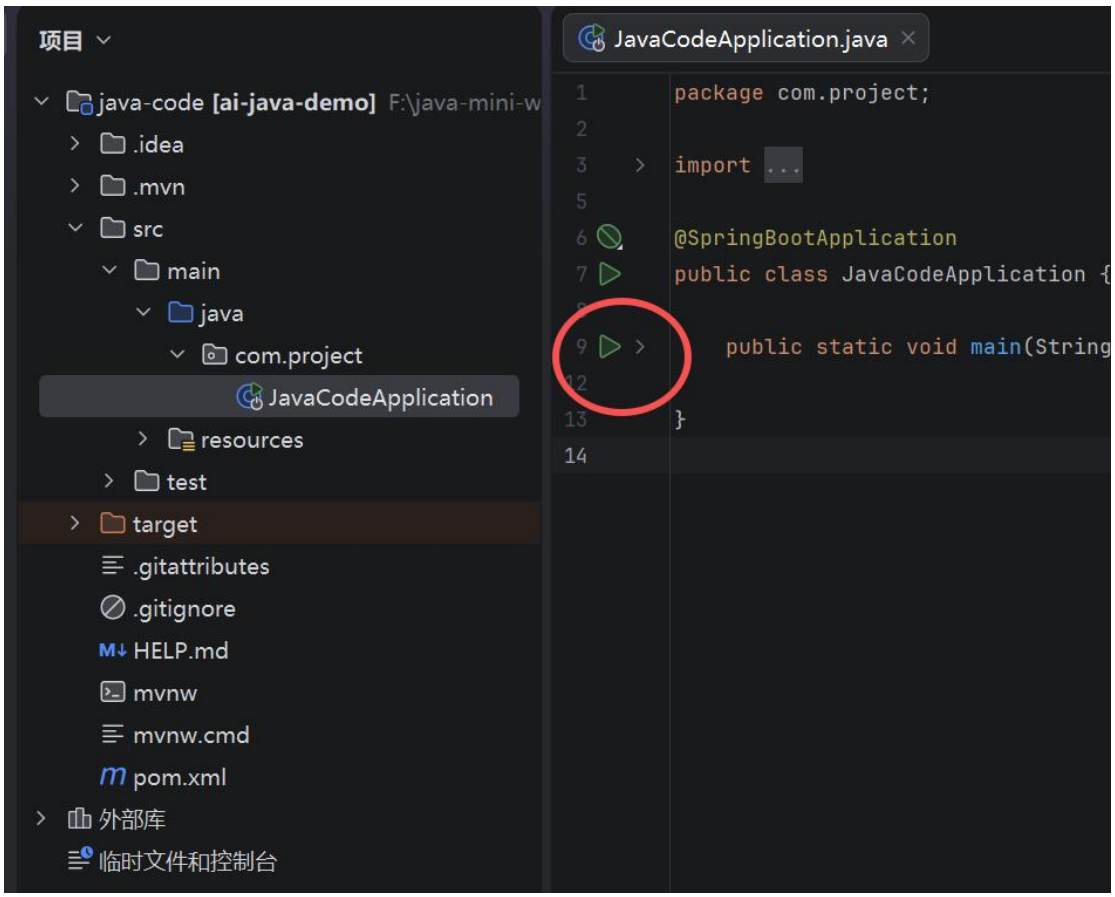
配置 maven

文件(F)
编辑(E)
视图(V)
导航(N)
代码(C)
重构
新建(N)
打开(O)...
最近的项目(R)
关闭项目(J)
远程开发...
设置(T)...
Ctrl+Alt+S
项目结构...
Ctrl+Alt+Shift+S
文件属性
本地历史记录(H)
全部保存(S)
Ctrl+S
从磁盘全部重新加载
Ctrl+Alt+Y
修复 IDE
使缓存失效...
管理 IDE 设置
新建项目设置
将文件另存为模板(L)...
导出
打印(P)...
省电模式
退出(X)

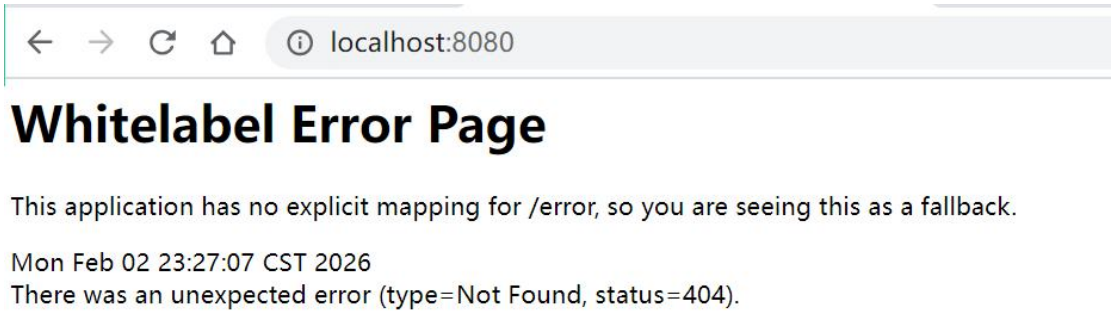


结束成功后项目就搭起来了

启动服务

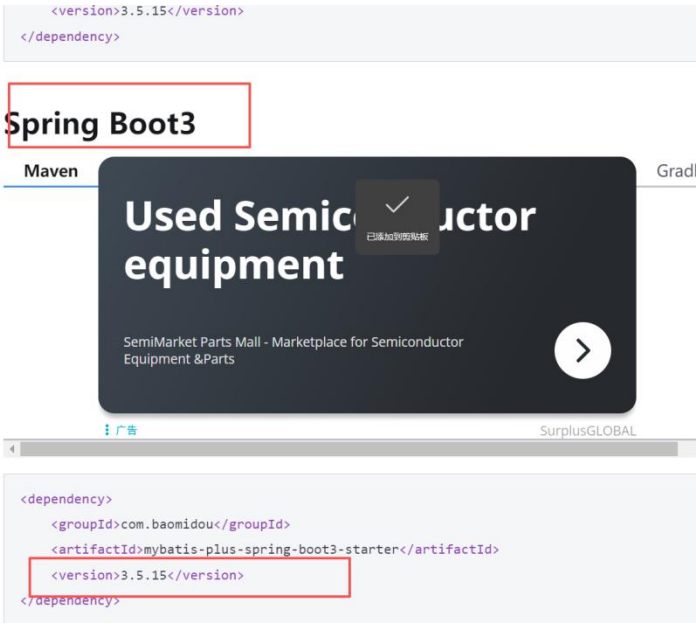


http://localhost:8080/



配置 mybatis-plus

https://baomidou.com/

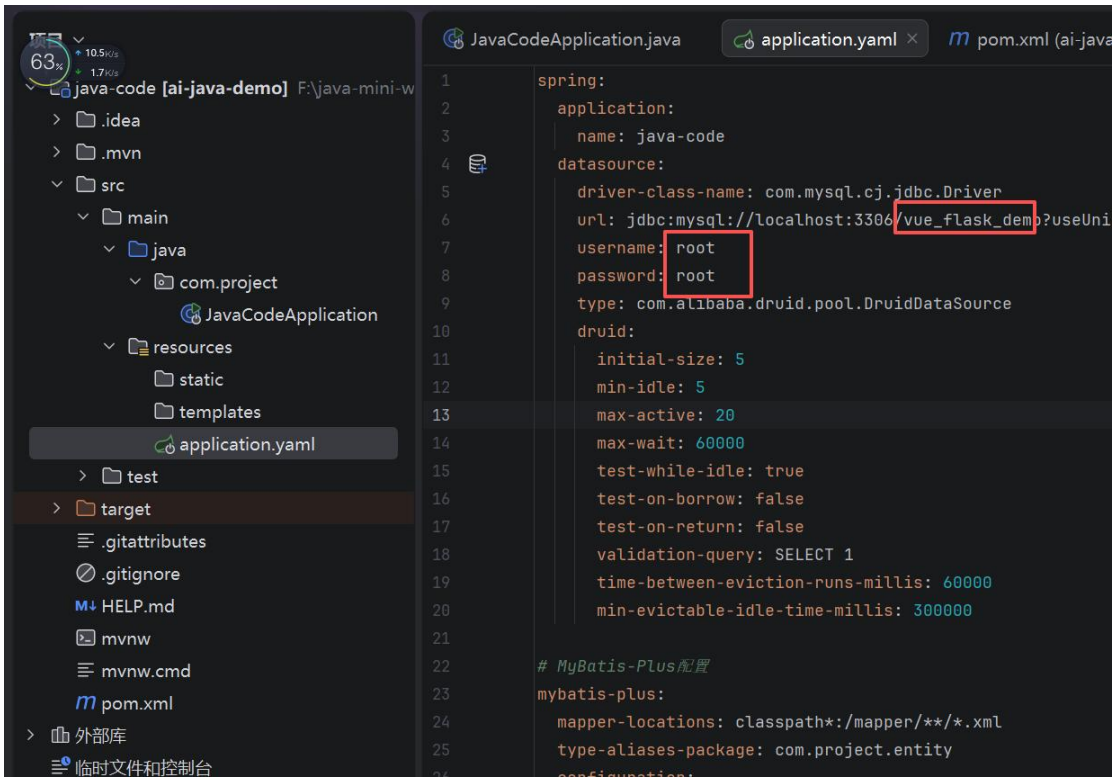


配置下 mybatis-plus3.5.15

数据库

如果没有给我们安装连接池，那就让 AI 做下 druid 连接池

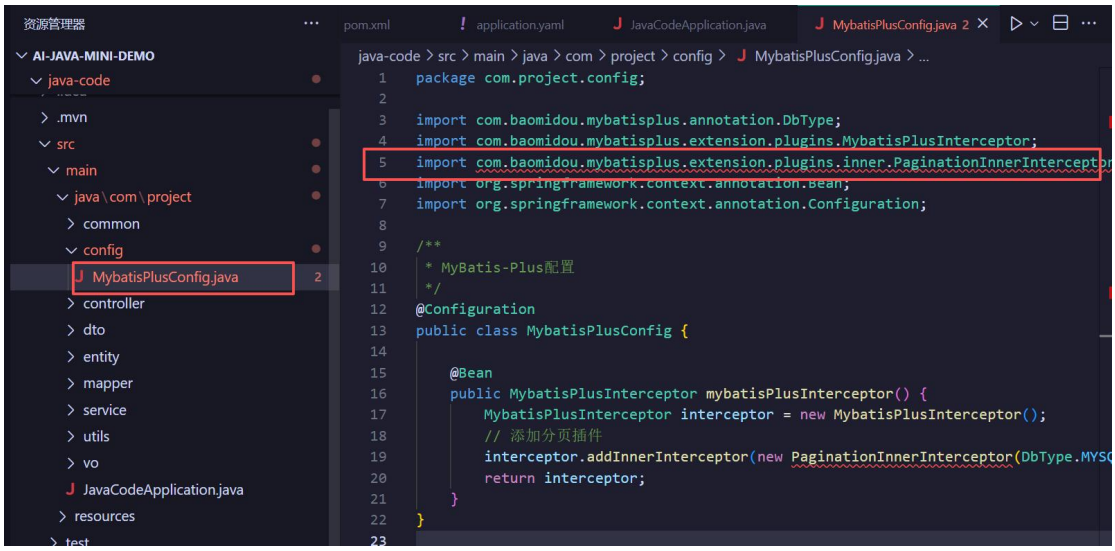
配置下数据库

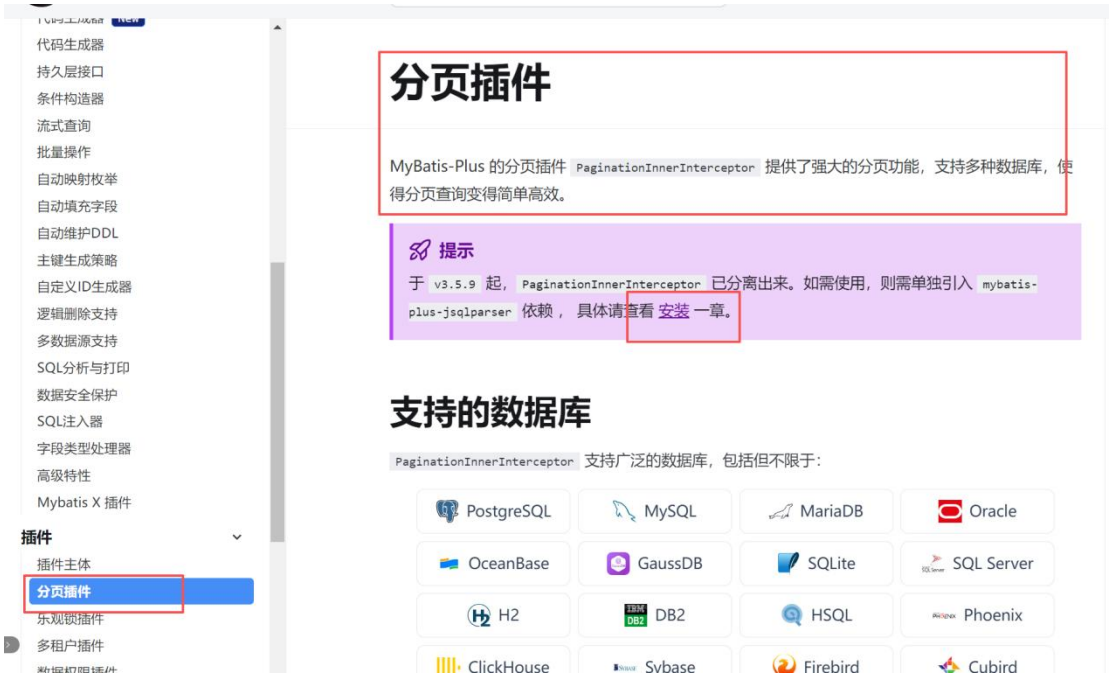


重新启动项目

创建下项目的目录结构，注意符合工程规范

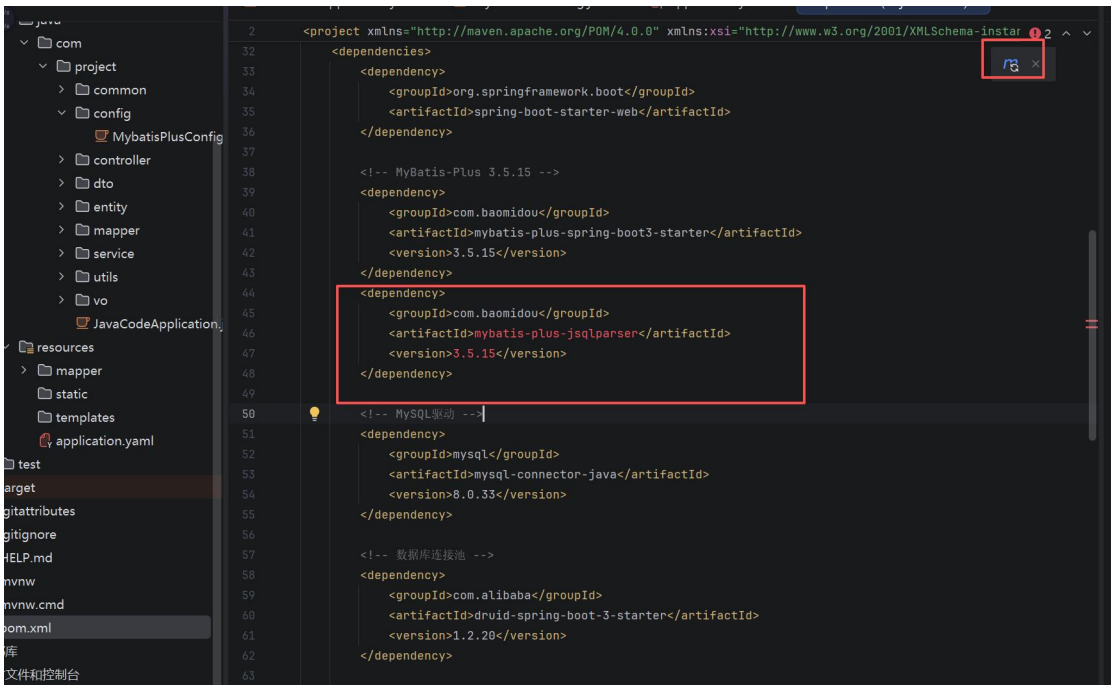
解决分页问题，有报错：





加上版本号

<version>3.5.15</version>

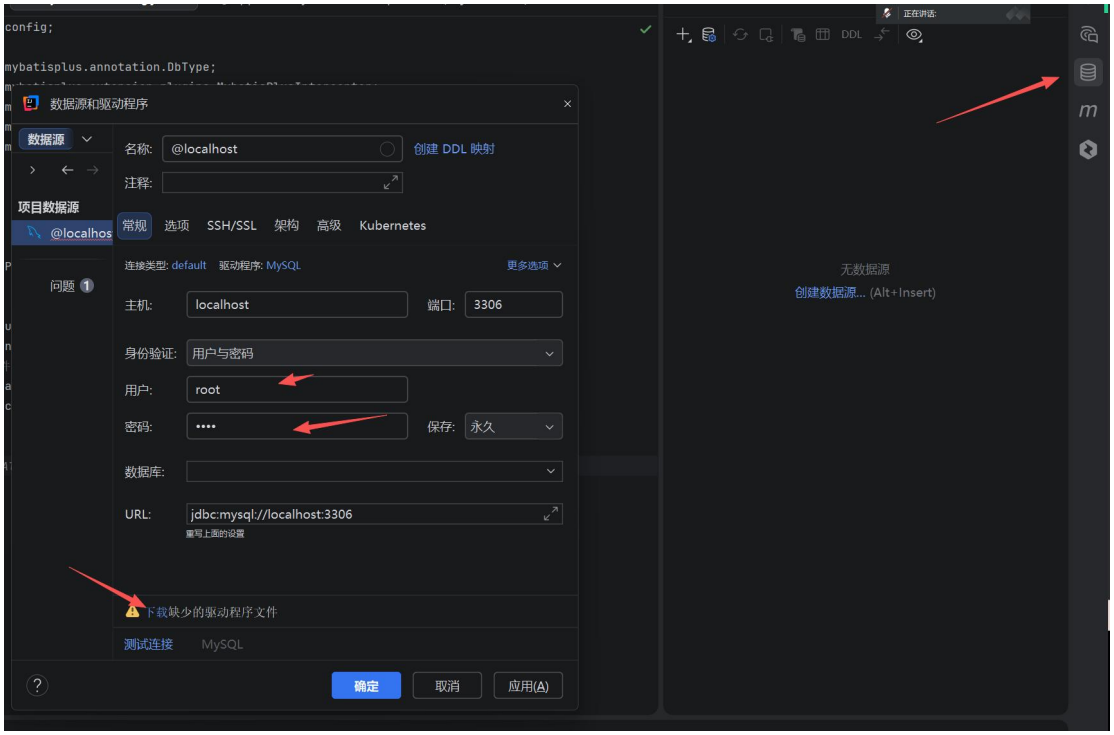



```
<!-- MyBatis-Plus 依赖 -->
        <dependency>
            <groupId>com.baomidou</groupId>
            <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
            <version>3.5.15</version>
        </dependency>
        <dependency>
            <groupId>com.baomidou</groupId>
            <artifactId>mybatis-plus-jsqlparser</artifactId>
            <version>3.5.15</version>
        </dependency>
```

写一个 HelloController，让我来测试下接口

controller 这个后端工程需要给前端后台管理和移动端小程序提供接口，帮我区分下接口文件，admin 文件下面是 pc 端后台管理的接口（接口文件前缀是 Admin），mini 文件下面是移动端后台管理接口（接口文件前缀是 Mini），只需要这俩个创建文件就可以，不需要实现其他的。

（可选）



测试连接

七、pc 端 后端开发

（1）提示词

1. 根据接口文档【API 接口文档.md】 和数据库【data1.sql】 ，完成【java-code】里后台管理系统后端接口的开发。
2.数据库字段如果使用 MySQL 保留关键字，SQL 查询时候需要注意。
3.使用安全框架 Spring Security 和 JWT 认证。密码先采用明文存储和验证。
4.pc 端后台管理 controller 相关的接口都放在 admin 中，移动端相关接口都放在 mini 中，便于权限管理和接口维护。请求路径前缀分别是/admin，/mini 。管理端 Controller 类名加上 Admin 前缀。Dto 或者 vo 也是通过前缀区分移动端和后台管理端。

（2）检查环境配置

检查数据库名称、账号、密码

（3）启动调试
IDEA 启动项目，报错信息给 AI

解决报错问题

(4)特别注意
后端开发好后一定要检查是符合接口要求，可以让 ai 重建检查一遍，否则可能出现与前端无法对接情况

提示词：

请检查当前接口实现的接口与接口文档@API.md 中的接口是否完全一致，如果不一致请修改，修改时注意前后连贯一 致。例如不能出现改了接口，服务层却还是旧的实现。

管理员登录接口一般是返回 token 就可以

生产环境建议： 使用 JWT Token 实现真正的登录验证，只有携带有效 Token 的请求才能访问受保护的接口。

七、pc 前端开发

注意：如果出现 axios.js 则修改前端工程文件名 axios.js 改为 request.js

【web-code 】 现在后端接口已经开发完成，请帮我根据接口文档 API【接口文档.md 】调用真实接口，不要模拟数据，直接调用后端真实接口实现。实现时注意检查每个模块功能前后一致连贯性, 对于不满足要求的代码可删除，需要添加新代码的地方添加新代码，可保留的代码可继续保留。

#API 接口文档.md 请严格按照接口文档的接口定义请求数据，检查 xxx 页面数据请求问题

图片上传

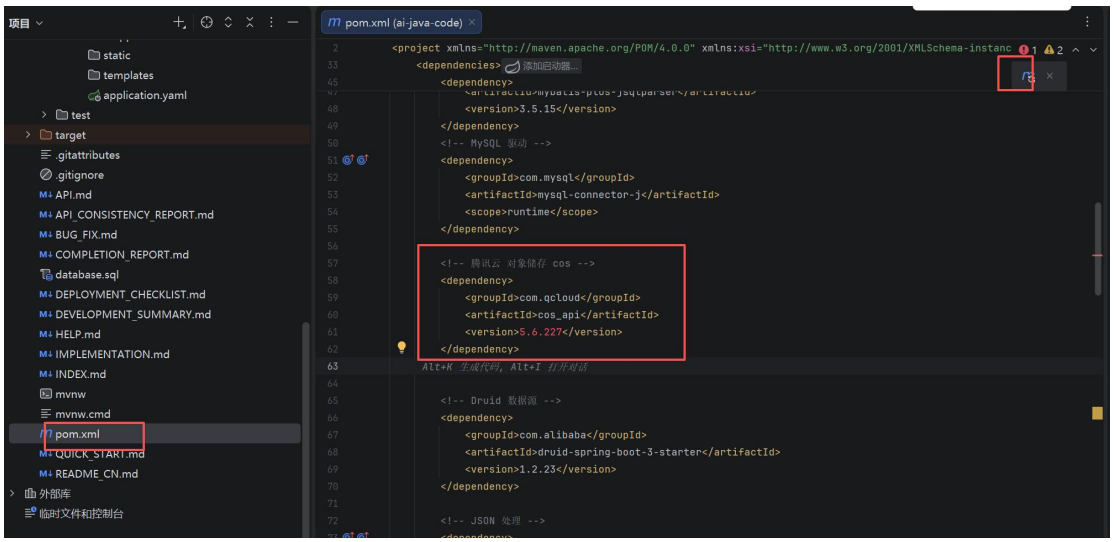
登录 https://cloud.tencent.com/

控制台搜索对象存储

ai-file -1300085904

ai-file-1300085904.cos.ap-guangzhou.myqcloud.com

添加依赖：




```
<dependency>
        <groupId>com.qcloud</groupId>
```

```
        <artifactId>cos_api</artifactId>
        <version>5.6.227</version>
</dependency>
```

提示词：

帮我在 java-code 里面创建一个上传文件的接口，采用腾讯云的 cos 来存储文件，上传成功后返回文件地址（比如：https://ai-file-1300085904.cos.ap-guangzhou.myqcloud.com/QQ%E6%88%AA%E5%9B%BE20220614122511.png）

```
// 1 初始化用户身份信息（secretId, secretKey）。
// SECRETID 和 SECRETKEY 请登录访问管理控制台 https://console.cloud.tencent.com/cam/capi 进行查看和管理
String secretId = System.getenv("secretId");//用户的 SecretId，建议使用子账号密钥，授权遵循最小权限指引，降低使用风险。子账号密钥获取可参见 https://cloud.tencent.com/document/product/598/37140
String secretKey = System.getenv("secretKey");//用户的 SecretKey，建议使用子账号密钥，授权遵循最小权限指引，降低使用风险。子账号密钥获取可参见 https://cloud.tencent.com/document/product/598/37140
COSCredentials cred = new BasicCOSCredentials(secretId, secretKey);
// 2 设置 bucket 的地域，COS 地域的简称请参见 https://cloud.tencent.com/document/product/436/6224
// clientConfig 中包含了设置 region, https(默认 http), 超时, 代理等 set 方法, 使用可参见源码或者常见问题 Java SDK 部分。
Region region = new Region("COS_REGION");
ClientConfig clientConfig = new ClientConfig(region);
// 这里建议设置使用 https 协议
// 从 5.6.54 版本开始，默认使用了 https
clientConfig.setHttpProtocol(HttpProtocol.https);
// 3 生成 cos 客户端。
COSClient cosClient = new COSClient(cred, clientConfig);
```

上传对象参考代码：

```
import java.io.File;

import com.qcloud.cos.COSClient;
import com.qcloud.cos.ClientConfig;
import com.qcloud.cos.auth.BasicCOSCredentials;
import com.qcloud.cos.auth.COSCredentials;
```

```
import com.qcloud.cos.exception.CosClientException;
import com.qcloud.cos.exception.CosServiceException;
import com.qcloud.cos.model.PutObjectRequest;
import com.qcloud.cos.model.PutObjectResult;
import com.qcloud.cos.region.Region;

public class PutObjectDemo {
    private static String secretId = System.getenv("SECRETID");
    private static String secretKey = System.getenv("SECRETKEY");
    private static String bucketName = System.getenv("BUCKET_NAME");
    private static String region = System.getenv("REGION");
    private static COSClient cosClient = createCli();

    public static void main(String[] args) {
        try {
            putLocalFileDemo();
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            cosClient.shutdown();
        }
    }

    private static COSClient createCli() {
        // 初始化用户身份信息(secretId, secretKey)
        COSCredentials cred = new BasicCOSCredentials(secretId,secretKey);
        // 设置 bucket 的区域, COS 地域的简称请参照
https://www.qcloud.com/document/product/436/6224
        ClientConfig clientConfig = new ClientConfig(new Region(region));
        // 生成 cos 客户端
        return new COSClient(cred, clientConfig);
    }

    private static void putLocalFileDemo() {
        String key = "abc/abc.txt";
        String localPath = "abc.txt";

        PutObjectRequest putObjectRequest = new PutObjectRequest(bucketName, key, new
File(localPath));
        try {
            PutObjectResult putObjectResult = cosClient.putObject(putObjectRequest);
            System.out.println(putObjectResult.getRequestId());
        } catch (CosServiceException cse) {
            cse.printStackTrace();
```

```
            } catch (CosClientException cce) {
                cce.printStackTrace();
            }
        }
}
```

application.yaml 里面配置参数：
存储桶的所属地域 region: ap-guangzhou
存储桶名称 bucket-name: ai-file-1300085904
SecretId: AKIDzmLhqmbDgx0sZ3KPHySjtqZHKGykJemy
secretKey: MYYAs67TprYO3Vp00mgokT1SiMX

根据 UploadController.java 补充下上传文件和图片的接口描述到接口文档里面 API.md

【src\views】`所有上传图片地方根据接口文档 【API.md】实现下上传图片功能

存储桶名称 ai-file-1300085904
SecretId: AKIDzmLhqmbDgx0sZ3KPHySjtqZHKGykJemy
secretKey: MYYAs6TpxrY3Vp00mgkTSiMX

直接修改 application.yaml

#Banner.vue 编辑和添加关联商品时候从数据库查询商品然后直接选择关联，如果涉及到新增接口，请将新增的接口补充到接口文档里面 #API.md

#views 有的页面数据列表宽度显示有问题

#views 轮播图管理、用户管理、管理员管理这三个页面的数据列表，操作列使用固定在右侧，确保表格在容器内正确显示

不断修正、测试

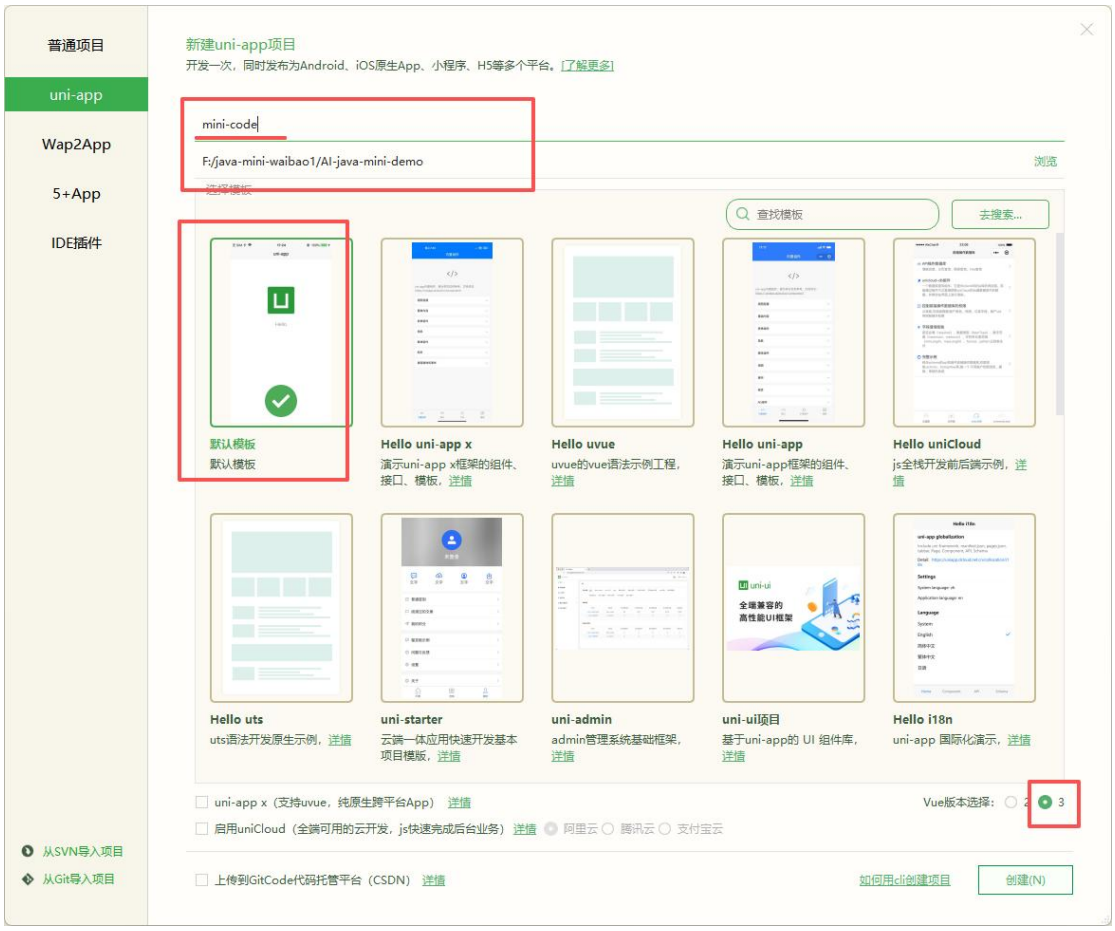
八、移动端前端 原型开发

https://www.dcloud.io/

https://mp.weixin.qq.com/

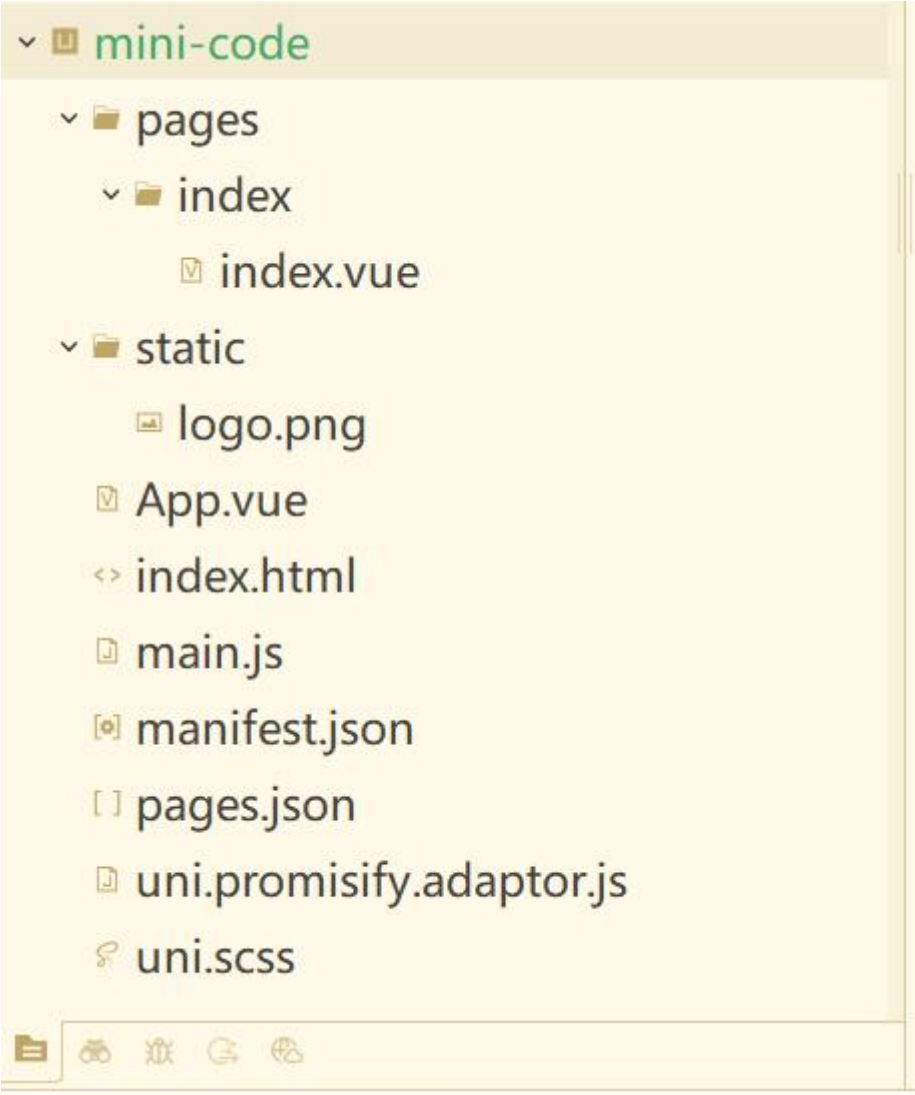
（1）创建工程

创建 uniapp 工程



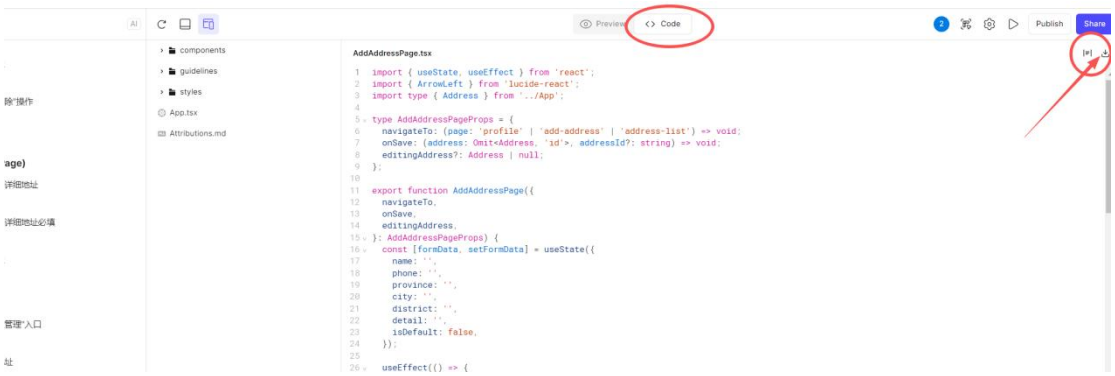
HBuilder X
当前项目没有使用源代码管理工具(git/svn)，是否继续替换?
注意："替换"将整体删除目录 F:/java-mini-waibao1/AI-java-mini-demo/mini-code 里的内容到回收站后，然后再复制新文件
改为合并(M)
继续替换(R)
取消(C)

结果


mini-code
pages
index
index.vue
static
logo.png
App.vue
index.html
main.js
manifest.json
pages.json
uni.promisify.adaptor.js
uni.scss

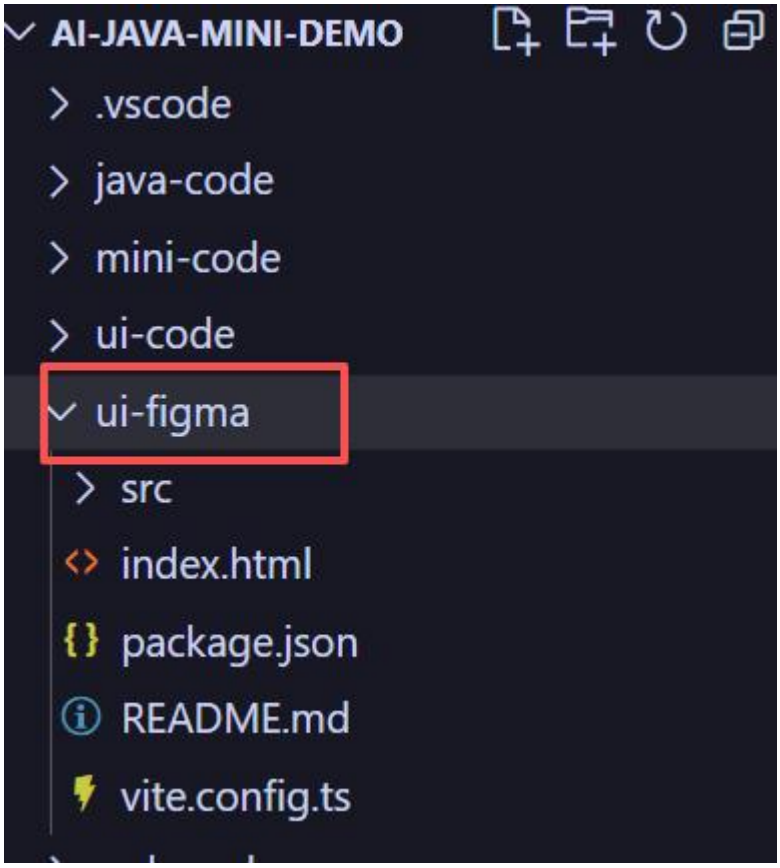
（2）复刻页面

下载 figma 原型图代码压缩包



工作区创建文件夹 ui-figma

解压放到 ui-figma 文件夹里面



使用 AI 开发工具复刻页面时候，首先将以下规则设置到项目规则里面：

Vue 组合式 API（如 ref, computed, onMounted, watch）→ 从 'vue' 导入 uni-app 页面生命周期（如 onLoad, onShow, onHide, onUnload, onPullDownRefresh）→ 从 '@dcloudio/uni-app' 导入 按钮尽量采用 view 来实现

Cursor、Trae、CodeBuddy 根据 figma 文件开发小程序页面

【ui-figma】 【mini-code】根据 Figma 原型图在 uniapp 工程中一比一复刻开发所有前端页面。要求如下： 1. 页面样式使用纯 CSS 实现，不使用 SCSS 2. 采用 Vue3 组合式 API 编写，使用<script setup>语法 3. 导入方式： - Vue 相关 API（ref, computed, onMounted, watch）从'vue'导入 - uni-app 生命周期（onLoad, onShow 等）从'@dcloudio/uni-app'导入 4. 按钮元素优先使用 view 组件实现 5. 确保页面布局、样式和交互效果与原型图完全一致 6. 代码结构清晰，遵循 Vue3 最佳实践 7. 所有页面都使用 pages.json 中配置的系统原生导航栏不需要再写自定义的 header 8. 移动端 uniapp 页面模拟数据字段注意与数据库【@数据库文件】保持一致，页面字段采用驼峰命名 请严格按照原型图的视觉效果和交互逻辑进行开发，保持高度还原度。

AI 检查

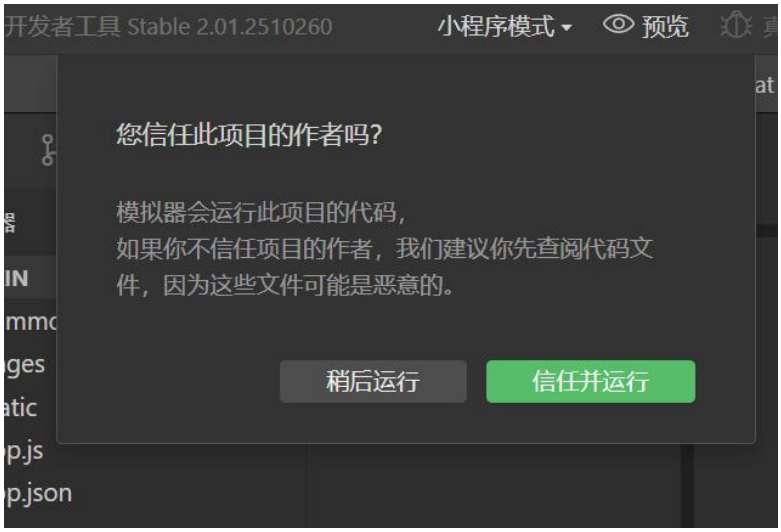
请检查下所有页面 Vue 组合式 API（如 ref, computed, onMounted, watch）→ 从 'vue' 导入 uni-app 页面生命周期（如 onLoad, onShow, onHide, onUnload, onPullDownRefresh）→ 从 '@dcloudio/uni-app' 导入

（3）运行项目

运行到浏览器

运行到微信小程序

文件(F) 编辑(E) 选择(S) 查找(I) 跳转(G) 运行(R) 发行(U) 视图(V) 工具(T) 帮助(Y)
运行到浏览器(B)
运行到内置浏览器(P)
运行到手机或模拟器(N)
运行到小程序模拟器(M)
运行到终端
停止微信开发者工具(W) - [mini-code]
微信开发者工具(E) - [mini-code] - 指定页面
百度开发者工具(B) - [mini-code]
百度开发者工具(N) - [mini-code] - 指定页面
支付宝小程序开发者工具(M) - [mini-code]
支付宝小程序开发者工具(D) - [mini-code] - 指定页面
抖音开发者工具(T) - [mini-code]
QQ开发者工具(N) - [mini-code]
360开发者工具(S) - [mini-code]
华为快应用开发者工具(U) - [mini-code]
快应用联盟开发者工具(K) - [mini-code]
快手小程序开发者工具(A) - [mini-code]
飞书小程序开发者工具(L) - [mini-code]
京东小程序开发者工具(J) - [mini-code]
小红书小程序开发者工具(X) - [mini-code]
鸿蒙元服务(O) - [mini-code]
运行时自动打开Vue Devtools(D)
运行时是否压缩代码(C)
运行设置(S)
AI死了么小程序端
AI死了么后台管理端
AI-mini-节能打卡小程序
AI-admin-节能打卡小程序
mini-code
pages
add-address
address-list
cart
category
home
index
login
order-confirm
order-list
product-detail
profile
register
static
logo.png
unpackage
App.vue
index.html
main.js
manifest.json
pages.json
uni.promisify.adaptor.js
uni.scss
"navigationBarBackgroundC
"backgroundColor": "#F8F8
"globalStyle": {
"navigationBarTextStyle":
"navigationBarTitleText":


开发者工具 Stable 2.01.2510260
小程序模式
预览
您信任此项目的作者吗?
模拟器会运行此项目的代码，
如果你不信任项目的作者，我们建议你先查阅代码文件，因为这些文件可能是恶意的。
稍后运行
信任并运行

（4）修正错误

将 manifest.json 中的 vueVersion 从 "2" 改为 "3"，现在项目将使用 Vue 3 编译。

下载图标 https://www.iconfont.cn/
按照 pages.json 图标路径创建文件夹，将 icon 放到文件夹里面。

然后让 AI 检查修复

检查下根据 Figma 原型图在 uniapp 工程中一比一复刻开发所有前端页面是否都复刻完成了 移动端 uniapp 页面模拟数据字段注意与数据库【@数据库文件】保持一致，页面字段采用驼峰命名，是否都完成了。

遇到一些问题解决不了可以尝试清除缓存或者关闭微信开发者工具重启

pages/category/index 这个页面添加到购物车的加号布局有问题 pages/login/index 、pages/register/index 登录注册页面输入账号密码输入框溢出页面了 pages/order/list 这个页面卡片溢出页面了 所有页面的>显示有问题，直接显示的是字符“>”

pages/order/list 这个页面订单列表 scroll-view 采用 view 来实现

pages/register/index、pages/login/index 这俩个页面的输入框高度有问题，遮挡了输入内容

pages/order/list 状态标签区域恢复要原来的 scroll-view 实现，只恢复状态标签区域

mini-code 微信小程序页面中的 > 直接显示为字符，而不是显示为 >

pages/category/index 加入购物车加号按钮显示到卡片的右下角

pages/address/index 添加地址弹窗里面的输入框溢出页面了

pages/profile/index 将 icon 里面的图标替换下待发货、待收货、已完成、售后图标

pages/profile/index 订单状态右上角的数量标签显示样式有问题，应该是按照圆形显示

pages/index/index 点击搜索跳入到新的页面，帮我实现下搜索页面功能，只需要实现前端页面就可以

九、移动端前端 接口文档生成

先备份下之前的 API.md 文档

（一）第 1 次提问：生成接口文档
现在请根据 uniapp 原型，生成【商城小程序移动端】后端接口文档并拼接到【API.md】后面。要求如下:
1 分析原型(原型在【/pages】目录下)中的功能模块，设计出移动端后端接口，并编写移动端接口文档。【API.md】里面已经有了 pc 端接口文档内容，对之前内容不要做任何改动，可以参考之前接口内容。
2 接口与原型前端对接，因此前后必须一致。
3 接口风格采用 RESTful 风格，接口必须符合工程规范，专业。
4 每个接口至少包括但不限于接口名称，接口功能，接口访问路径，接口请求方式，接口的请求传递参数，接口返回数据，并要求有使用示范案例。注意一定要专业实用(如果有我没提到但属于专业也加上)。
5.购物车、地址管理通过缓存实现，不需要接口实现。
6.移动端小程序接口路径格式应为 /api/mini/{模块}/{操作}，这样可以清晰区分 PC 端后台管理和移动端的接口。

（二）第 2 次提问：修正文档

Ask 模式：

认真检查一下当前的【@接口文档.md】是否完全支持移动端 uniapp 原型【/pages】中的所有功能，和【@database.sql】数据库的字段匹配一致,因为后端将严格按照接口实现，如果接口不完善，将导致前后无法对接。数据库没有的字段在 uniapp 原型中不需要实现对应的功能。

注意：不需要他在文档里面要求后端怎么实现，只需要专注写明接口文档就可以了。

不断检查修正文档

API.md 完善下 API 文档内容： 删除商品 description 该字段 缺少图片上传接口：用户头像上传、评价图片上传 订单状态转换逻辑：付款后是待发货状态，点击收货后是已完成状态 搜索结果可能需要按销量、价格排序

API.md 用户头像上传，评价图片上传这俩个接口放到商城小程序移动端接口文档部分

待支付 → 待发货: 用户完成支付后自动变更
待发货 → 待收货: 商家发货后由管理员手动更新
待收货 → 已完成: 用户点击确认收货后变更
待支付 → 已取消: 用户取消订单
待发货/待收货 → 退款/售后: 用户申请退款或售后

（三）第 3 次提问：修正文档 驼峰命名 从接口中发现缺少数据库情况下补充数据库和数据库字段

@接口文档.md 我认真检查了一下接口文档，请求参数名字中有很多下划线命名，而后台返回的是驼峰命名，可能导致前后无法对接，请把接口文档中参数(包括请求参数和返回参数)的名字由下划线命名改为驼峰命名。

【API.md】根据接口文档中的评价接口帮我补充下评价数据库表【database.sql】

【database.sql】用户表添加 phone、email、gender、birthday 字段

十、移动端前端 后端开发

（1）提示词

根据接口文档【API 接口文档.md】 移动端接口内容 和数据库【data1.sql】 ，完成【java-code】里移动端后端接口的开发，移动端接口 controller 统一放在【mini】目录里面，每个 controller 专注于单一业务领域。 移动端 Controller 类名加上 mini 前缀。

(2)检查
后端开发好后一定要检查是符合接口要求，可以让 ai 重建检查一遍，否则可能出现与前端无法对接情况

提示词：
ASK 模式：

请检查【mini】当前接口实现的接口与接口文档【@API.md】 中移动端的接口是否完全一致，如果不一致请修改，修改时注意前后连贯一 致。例如不能出现改了接口，服务层却还是旧的实现。

十一、移动端前端 前端开发

（1）提示词

1【mini-code 】 现在移动端后端接口已经开发完成，请帮我根据接口文档 API【接口文档.md 】 调用真实接口，不要模拟数据，直接调用移动端后端真实接口实现，完成移动端的接口请求开发。

2 实现时注意检查每个模块功能前后一致连贯性, 对于不满足要求的代码可删除，需要添加新代码的地方添加新代码，可保留的代码可继续保留。

3 购物车功能和用户地址管理通过缓存实现，不需要对接接口实现。如果页面功能缺少对应接口定义的话，不删除任何页面功能，不修改接口文档内容，给我总结提示就可以。

#API 接口文档.md 请严格按照接口文档的接口定义请求数据，检查 xxx 页面数据请求问题

（2）不断修正、测试

Vue 组合式 API（如 ref, computed, onMounted, watch）→ 从 'vue' 导入 uni-app 页面生命周期（如 onLoad, onShow, onHide, onUnload, onPullDownRefresh）→ 从 '@dcloudio/uni-app' 导入

pages/index/index 点击轮播图没有跳转到绑定的对应商品详情页面

pages/product/detail 商品详情没有显示详情图数组图片 detailImages

pages/product/detail
规格这块后台返回的数据格式是下面的格式，请按照这种格式重新优化下规格选择功能
[{"name":"颜色","value":"黑色"},{"name":"颜色","value":"蓝色"},{"name":"尺码","value":"39"},{"name":"尺码","value":"40"}]

Java-code 如何调大上传文件的大小

现在后台是有对应的分类 ID 为 4 的数据的，但是接口没有请求到，请检查下是不是后台接口有问题

如果后端接口修改更新了，请帮我将接口更新内容补充更新到接口文档 API.md，让接口文档准确反映后端实际的接口信息。

商品详情页面和分类页面添加商品到购物车成功后，购物车页面 pages/cart/index 并没有立马显示购物车里面的商品列表，购物车是通过缓存来实现的

用户添加收货地址后再提交订单页面没有显示默认的地址，用户管理地址采用缓存来实现

pages/order/confirm 弹窗显示未登录或 token 实现应该直接跳转到登录页面去

pages/order/confirm 未登录或 Token 无效时候没有跳转到登录页面

pages/order/confirm 点击提交订单时候，在请求发起之前 orderItems 这个字段里面的商品补充下商品的封面图和标题

pages/order/confirm 点击提交订单时候弹出虚拟支付框，然后有取消按钮。输入 6 位数密码后（666666），如果输入正确密码，然后提交订单（待发货状态），如果取消，提交订单（待支付状态）。

pages/order/list 点击订单进入到订单详情里面

pages/order/list 订单时间没有格式化，订单详情页面也是，商品详情页面评价也是没有格式化时间

java-code　按照 API.md　接口文档里面的新增接口实现下接口开发

✅ 4.6 支付订单接口 (POST /api/mini/order/pay/{id}) 功能说明 请求参数 返回数据 使用示例

✅ 4.7 删除订单接口 (DELETE /api/mini/order/delete/{id}) 功能说明 请求参数 返回数据 使用示例

pages/order/list 点击删除订单没有删除掉数据

pages/product/detail 评价这里没有显示头像信息，修复下这个问题，如果接口信息发生变化请对应的更新下接口文档 API.md

pages/order/confirm 订单支付成功后，订单详情里面对应的商品库存-1，销量+1，
pages/order/list 这个页面点击支付，支付成功后订单详情里面对应的商品库存-1，销量+1

如果接口信息发生变化请对应的更新或者补充下接口文档 API.md

十二、Pc 端优化

web-code
　所有管理页面的列表页面的创建时间字段格式化显示下，现在显示是 2026-02-08T03:12:28 这种格式

订单管理里面点击详情弹窗里面显示下订单里面的商品数据，数据库存储格式比如某一条数据：[{"productId":1,"quantity":2,"selectedSpecs":{"内存":"16G","颜色":"黑色"},"coverImage":"https://ai-file-1300085904.cos.ap-guangzhou.myqcloud.com/images/67be7457-acf4-43c4-a407-30de9c970581.png","productName":"联想拯救者Y7000P","price":"7500"},{"productId":3,"quantity":1,"selectedSpecs":{"颜色":"黑色","尺码

":"39"},"coverImage":"https://ai-file-1300085904.cos.ap-guangzhou.myqcloud.com/images/43bce210-60d6-4d3f-a72d-e2cbbc847067.png","productName":"达芙妮爆款休闲运动鞋.谁懂!达芙妮这鞋美疯","price":"288"}]

API.md 9.1 获取首页统计数据 修改下订单总数这块部分，改成只查询当天的订单总数，请修改对应的后端代码和更新接口文档

添加统计图功能

Home.vue

添加俩个 Echarts 统计图，一个是商品评价词云图、一个是商品销量前 10 统计图（看什么图比较合适），采用模拟数据就可以，只需要实现页面就可以。

API.md

添加下后台管理的

Home.vue

页面的商品评价关键词、商品销量 TOP10 接口内容，然后根据接口文档开发下这俩个功能

Xxx

学习 1：

Spring Security 和 jwt 关系
一句话总结： Spring Security 是安全框架（系统），JWT 是认证令牌（工具），Spring Security 使用 JWT 来完成无状态的身份认证。就像：机场安检系统（Spring Security）使用登机牌（JWT）来验证乘客身份。
JWT 是 Spring Security 的一种认证方式
Spring Security 本身支持多种认证方式：

- [x] 用户名密码
- [x] Session
- [x] OAuth2
- [x] JWT ← 你的项目使用这个

Spring Security 和 JWT 是不同层面的技术，它们之间是互补关系，不是替代关系。让我用项目中的代码来详细解释：

它们的定位
Spring Security - 安全框架（保护神）
Spring Security 是一个完整的安全框架，负责：

身份认证（你是谁？）
授权控制（你能做什么？）
防止各种攻击（CSRF、XSS 等）
管理安全相关的所有功能
在你的项目中，Spring Security 是安全守门员：

Spring Security 做的事情：

- ☑ 决定哪些接口可以公开访问
- ☑ 决定哪些接口需要认证
- ☑ 过滤所有 HTTP 请求
- ☑ 管理安全上下文（SecurityContext）

JWT - 认证令牌（通行证）

JWT 是一种令牌格式，用于在各方之间安全传递信息。

在你的项目中，JWT 是用户的身份证明：

JWT 做的事情：

- ☑ 存储用户 ID
- ☑ 存储过期时间
- ☑ 数字签名防篡改
- ☑ 自包含所有认证信息

类比理解

Spring Security = 机场安检系统

负责整个安全流程

决定哪些人可以登机

管理所有的安检口

JWT = 乘客的登机牌

包含乘客身份信息

包含航班信息

有防伪标记

JwtAuthenticationFilter = 安检人员

检查登机牌是否有效

验证登机牌上的签名

如果有效，告诉安检系统"这个人可以通过"

方面	Spring Security	JWT	关系
类型	安全框架	令牌格式	框架 vs 数据
职责	统一管理安全	存储认证信息	管理者 vs 证据
作用域	整个应用安全	身份认证	宏观 vs 微观
是否必需	不是（可不用）	不是（可用其他方式）	互补使用
在你的项目中	✅ 使用	✅ 使用	JWT是Spring Security的一种认证方式

一句话总结： Spring Security 是安全框架（系统），JWT 是认证令牌（工具），Spring Security 使用 JWT 来完成无状态的身份认证。

就像：机场安检系统（Spring Security）使用登机牌（JWT）来验证乘客身份。

学习 2：

http://localhost:8080/api/v1/banner/list 浏览器里面输入地址后这个提示未授权
http://localhost:8080/api/v1/mini/banners 浏览器里面输入地址后这个返回了数据

为什么会出现这种情况
1. /api/v1/banner/list - 管理后台接口
这个接口属于管理后台，需要 JWT 认证才能访问。从 Spring Security 配置可以看到：

SecurityConfig.javaL36-L39
.requestMatchers("/api/v1/admin/login", "/api/v1/mini/**").permitAll()
.anyRequest().authenticated()
解释：

只有 /api/v1/admin/login 和 /api/v1/mini/** 路径允许公开访问
其他所有路径都需要认证，包括 /api/v1/banner/list
在浏览器中直接访问，没有携带 JWT token，所以提示未授权
2. /api/v1/mini/banners - 小程序端接口
这个接口属于小程序端，被配置为公开访问，不需要 JWT 认证。

MiniController.javaL24-L29
@GetMapping("/banners")

```
public Result<?> getBanners() {
    LambdaQueryWrapper<Banner> wrapper = new LambdaQueryWrapper<>();
    wrapper.orderByDesc(Banner::getCreateTime);
    return Result.success(bannerService.list(wrapper));
}
```

为什么有两套接口？
这是一个典型的多端应用设计：

端	路径前缀	用途	是否需要认证
管理后台	/api/v1/banner/*	管理员操作（增删改查）	✅ 需要 JWT
小程序端	/api/v1/mini/banners	用户浏览（只读）	❌ 公开访问

管理后台：需要登录后才能操作，保证数据安全 小程序端：用户浏览轮播图不需要登录，提升用户体验

如何测试管理后台接口？
如果你想在浏览器中测试 /api/v1/banner/list，需要：

先登录获取 token：

code
POST http://localhost:8080/api/v1/admin/login
Body: {"account": "admin", "password": "123456"}
使用 token 访问管理接口： 在浏览器开发者工具中手动添加 Authorization 头：

code
Authorization: Bearer <你的 token>
或者使用工具如 Postman、Apifox 等，更容易管理认证信息。

总结
这是设计预期的行为，不是 bug：

✅ /api/v1/mini/banners 公开访问 - 正确
✅ /api/v1/banner/list 需要认证 - 正确
❌ 在浏览器中直接访问需要认证的接口会提示未授权 - 这是正常的
建议：使用前端管理系统（/web-code）或 API 测试工具来访问需要认证的管理接口

学习 3
DTO（请求参数）
VO（响应数据）